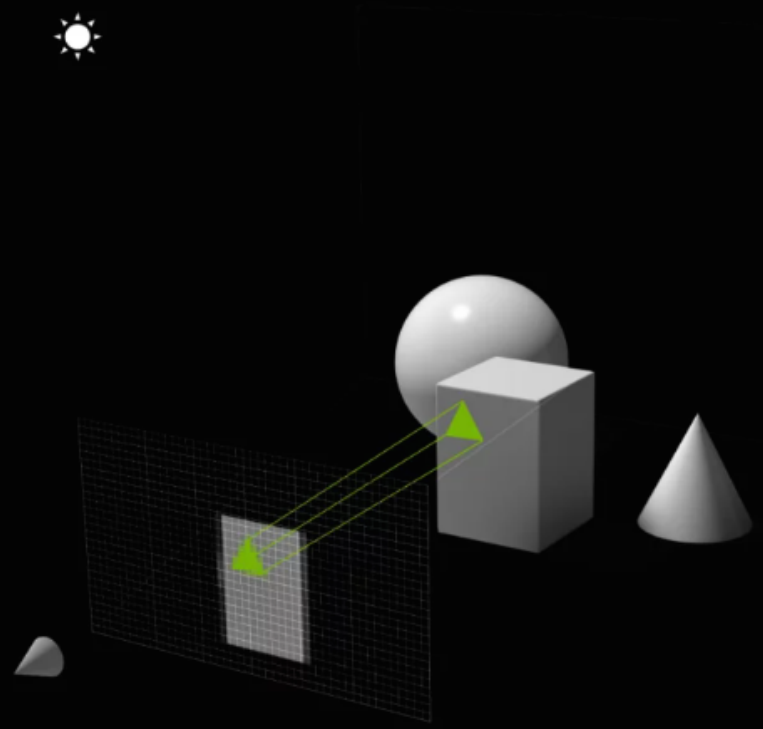
Rasterization

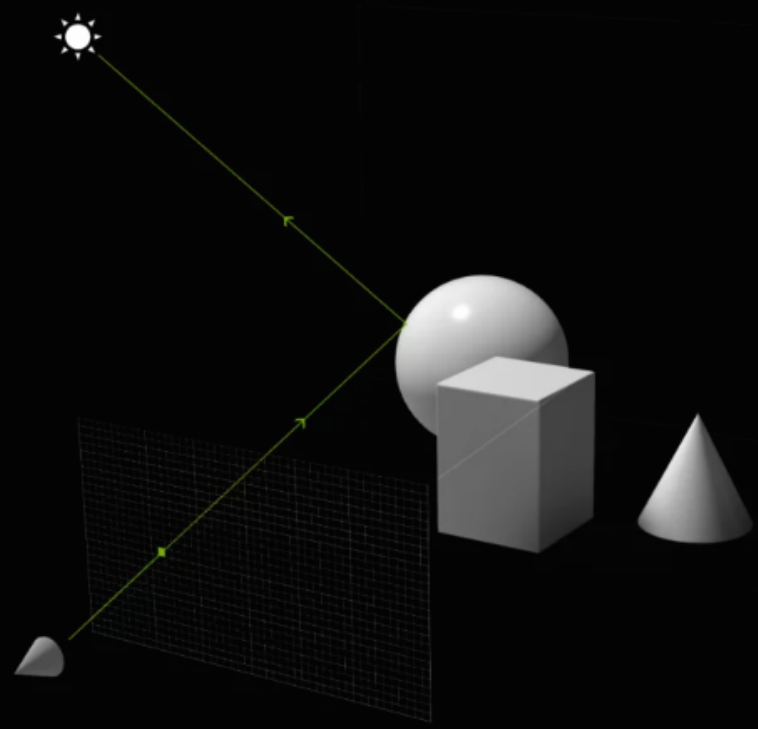
DongJoon Kim

| Phone : +82-2-940-8461
| Email : dongjoonkim@kw.ac.kr

THE HOLY GRAIL OF GRAPHICS



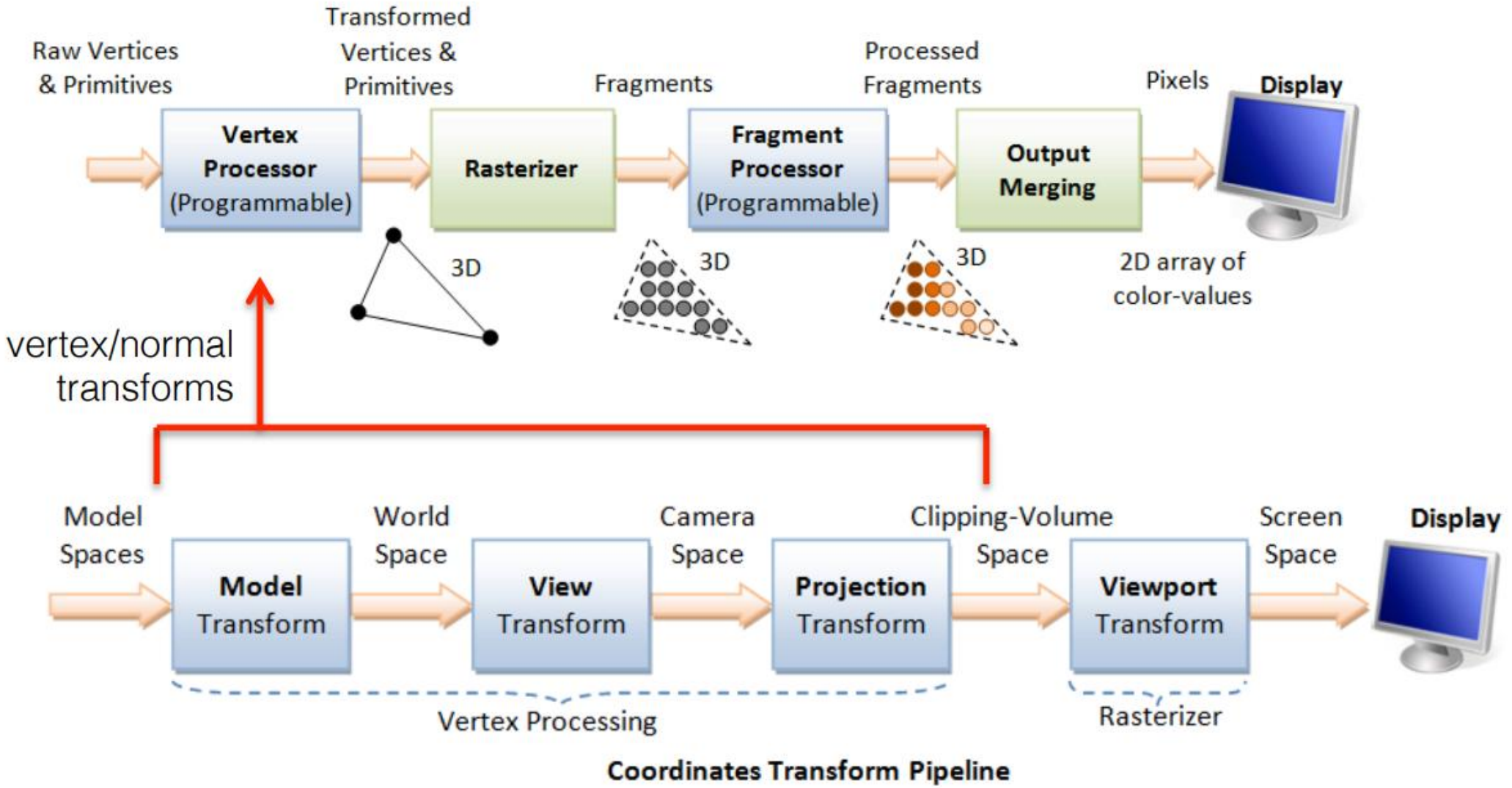
RASTERIZATION



RAY TRACING

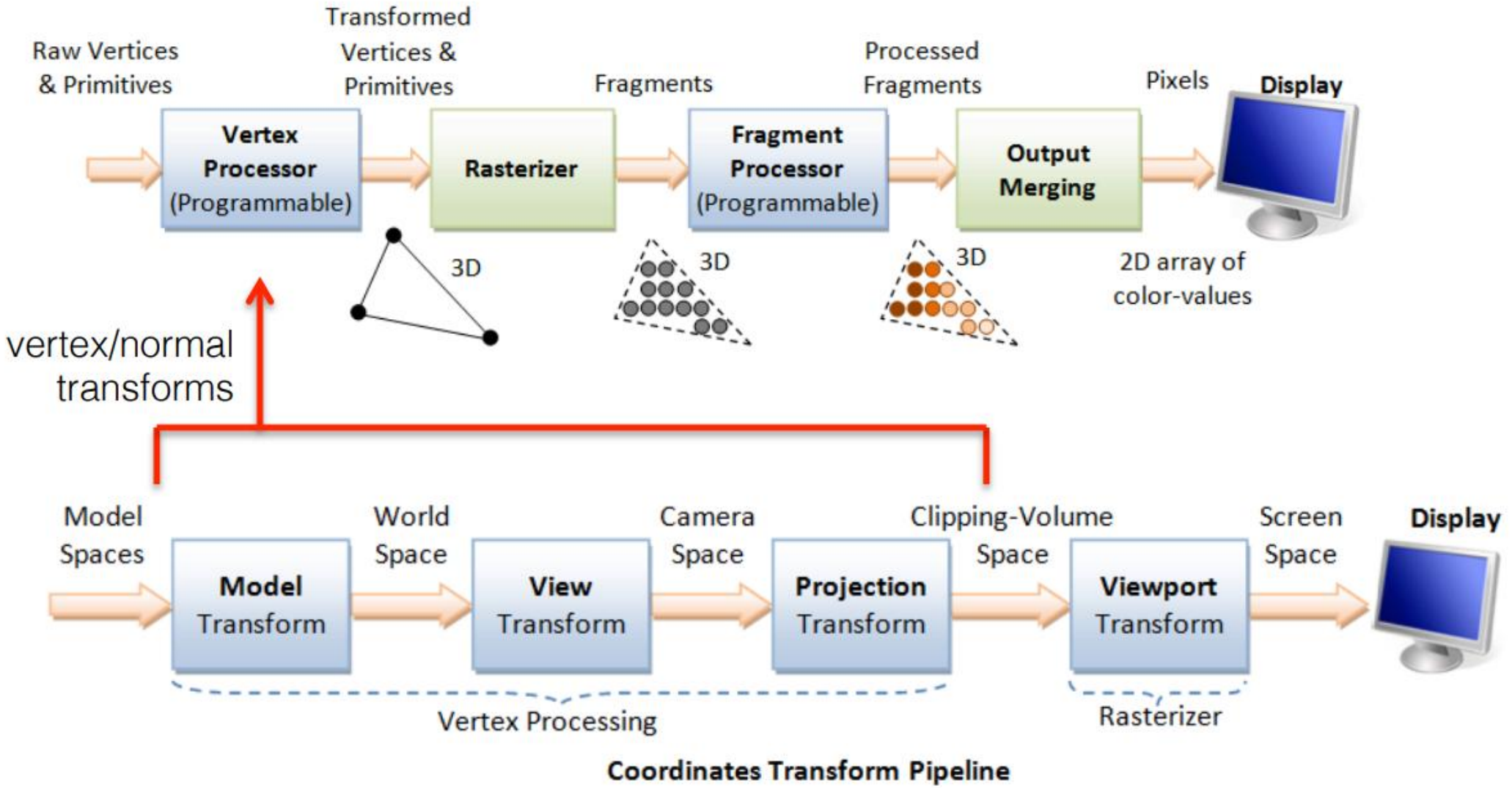
In Ray Tracing, light behaves in the game as it would in real life – Image: Nvidia

Rasterization-based Graphics Pipeline



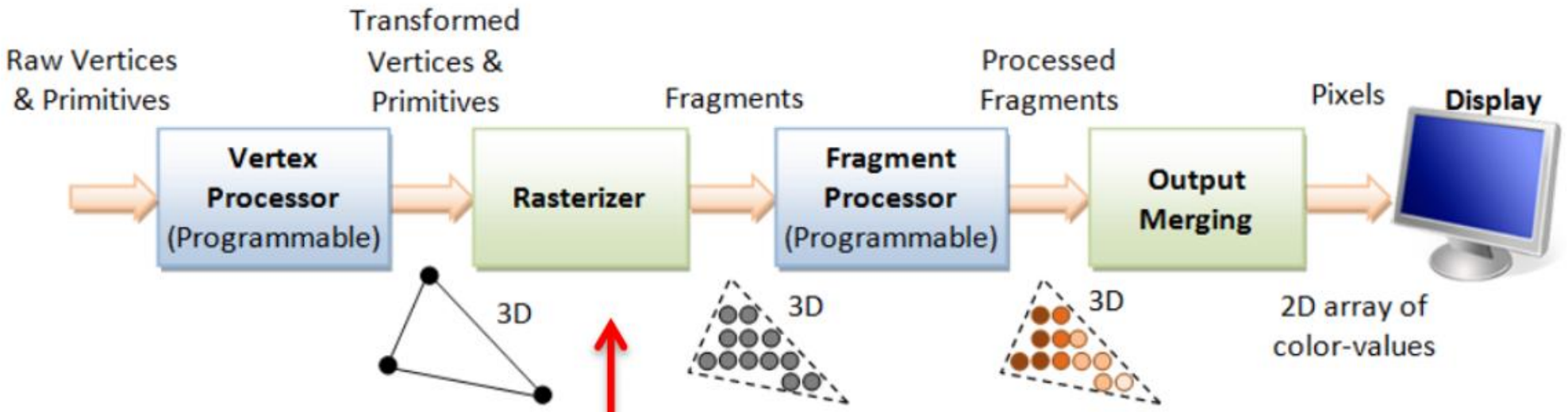
https://www.ntu.edu.sg/home/ehchua/programming/opengl/CG_BasicsTheory.html

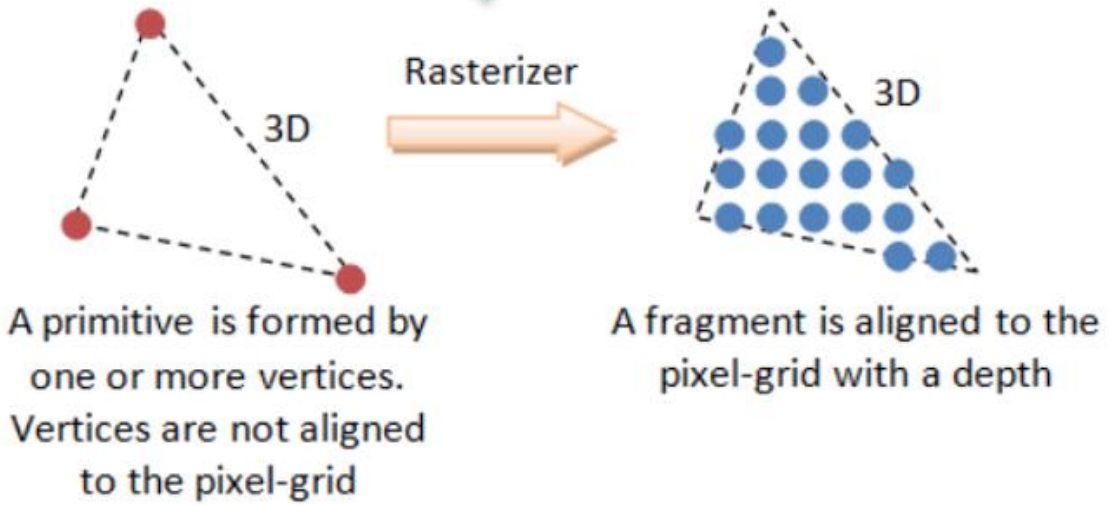
Review: Graphics Pipeline



https://www.ntu.edu.sg/home/ehchua/programming/opengl/CG_BasicsTheory.html

Rasterization





Purpose:

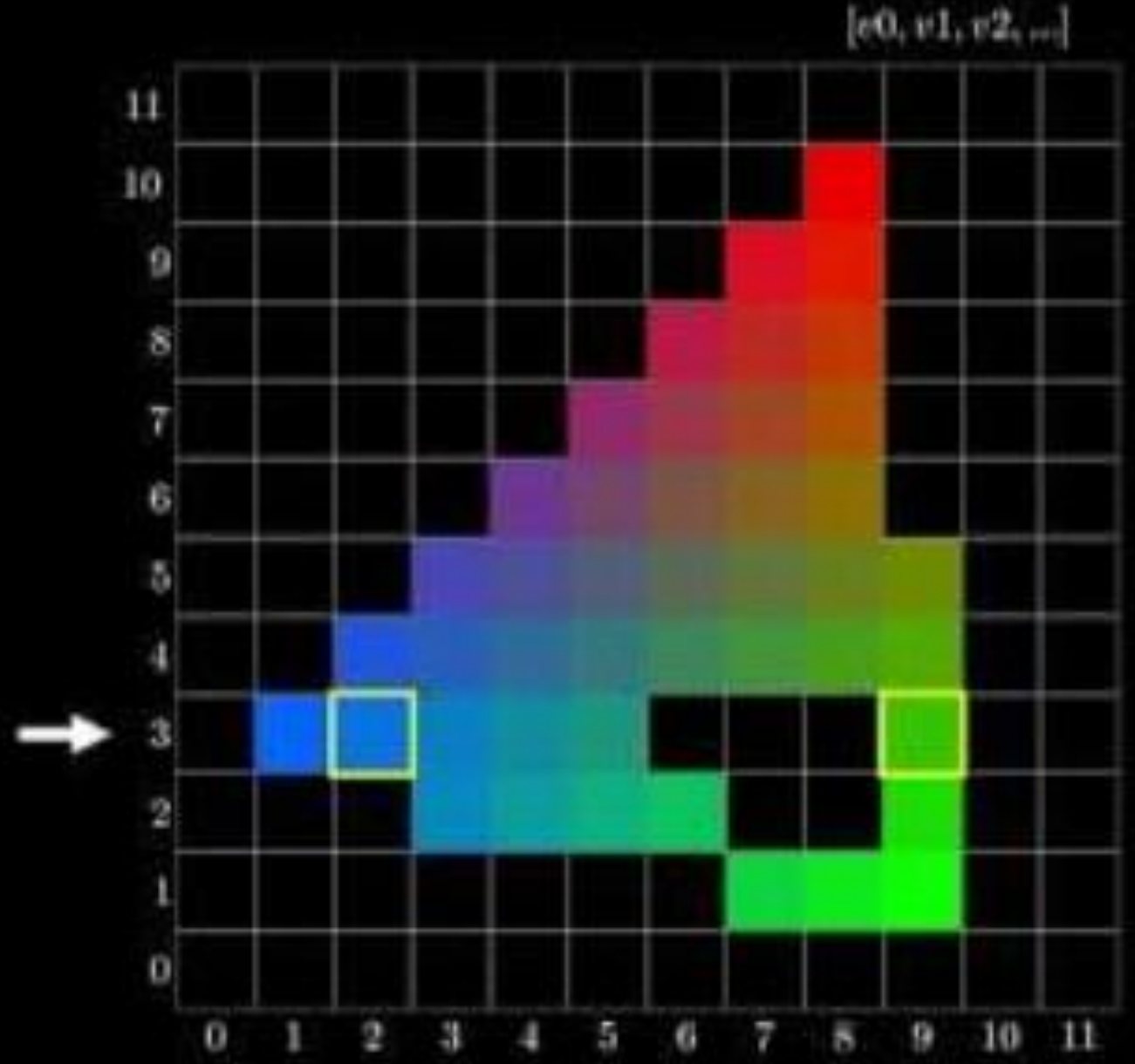
1. determine which pixels are inside the triangles
2. interpolate vertex attributes (e.g. color) to all fragments

https://www.ntu.edu.sg/home/ehchua/programming/opengl/CG_BasicsTheory.html

Scanline algorithm

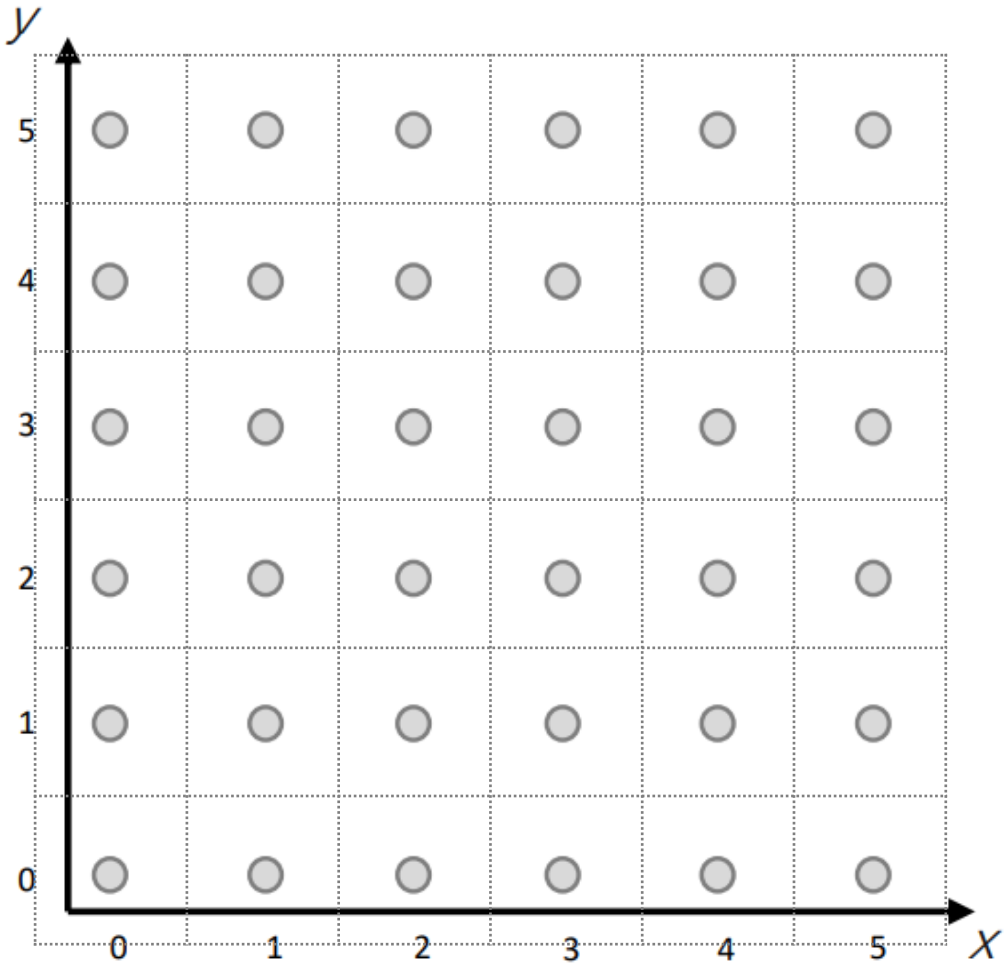
$all\_x\_on\_y[3] = [1, 1, 2, 9]$

$drawLine(x1, x2, y)$

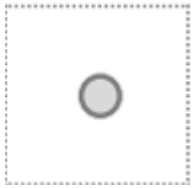


Rasterization

Rasterization



- grid of 6x6 pixels

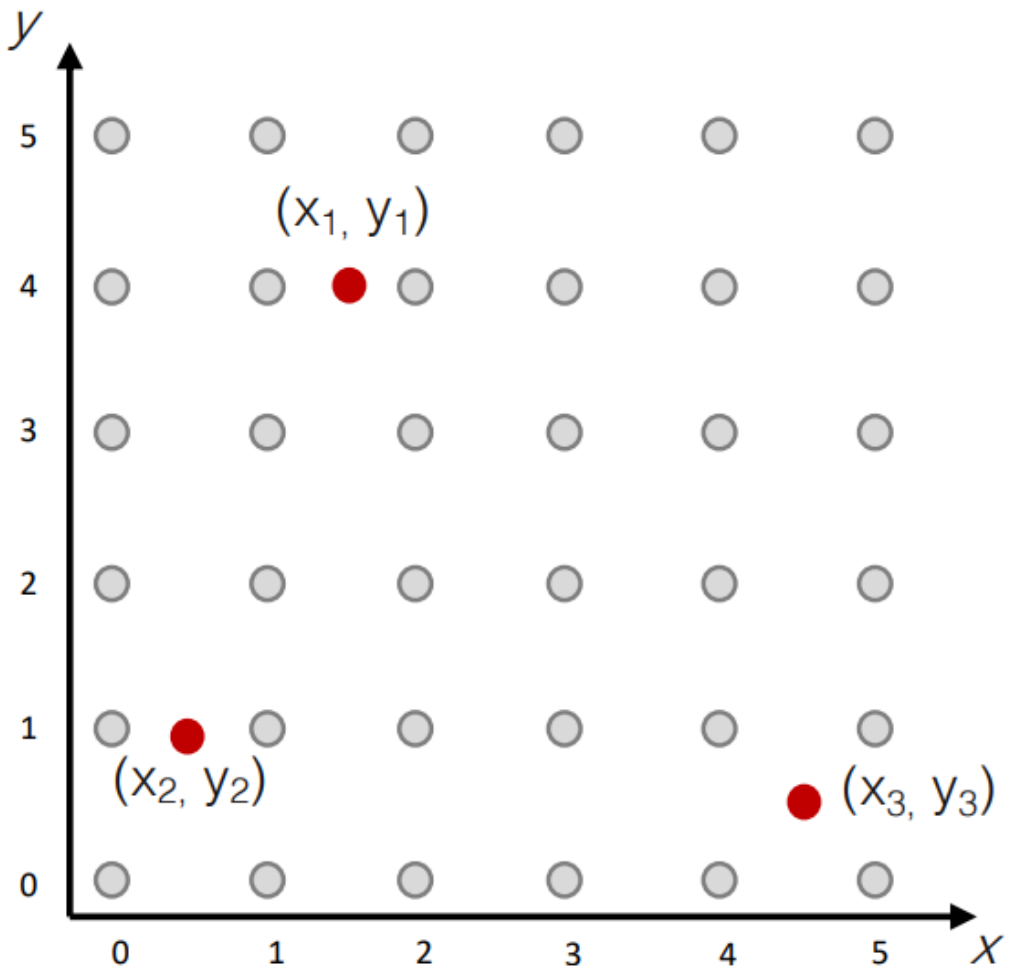


area of a pixel (circle indicates the position of the pixel)
In many case, ‘pixel’ and ‘fragment’ can be used interchangeably

Let’s assume vertices of a triangle are projected onto the grid

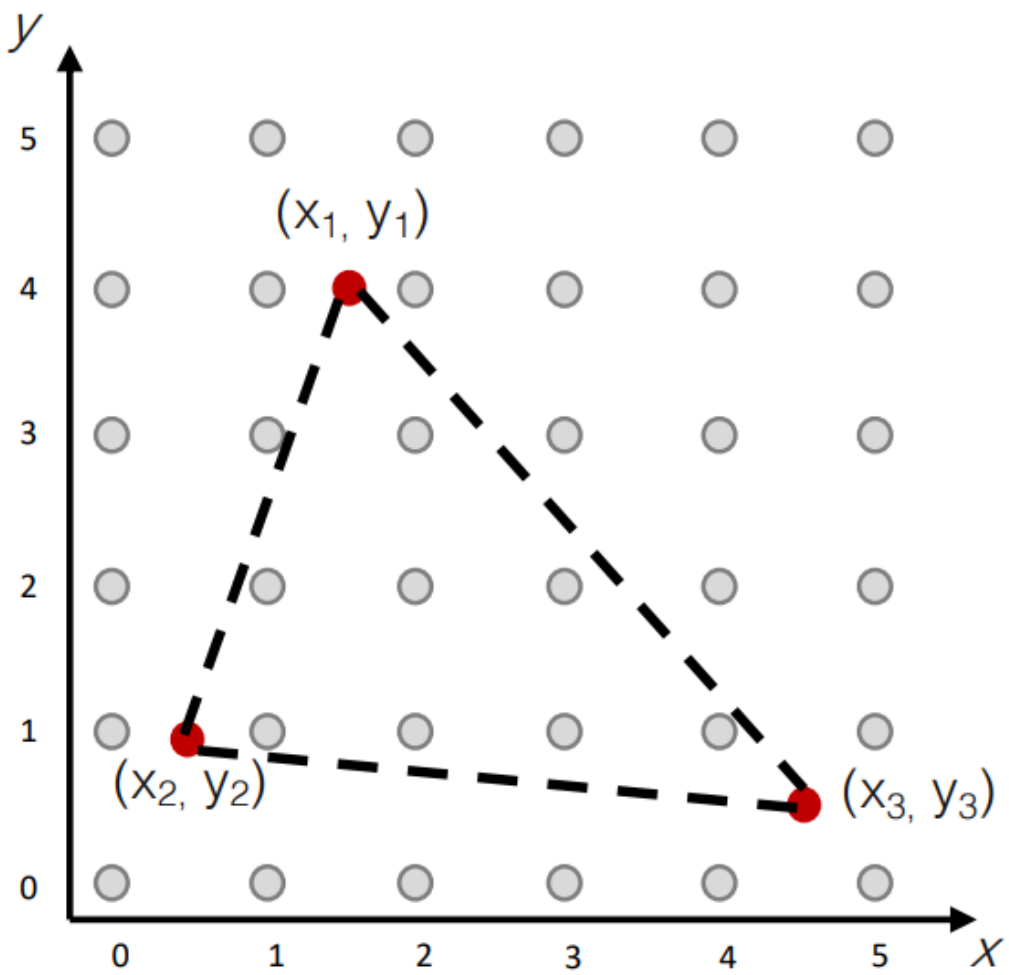
Refers to: Stanford EE267 course

Rasterization



- grid of 6x6 pixels
- 2D vertex positions after transformations

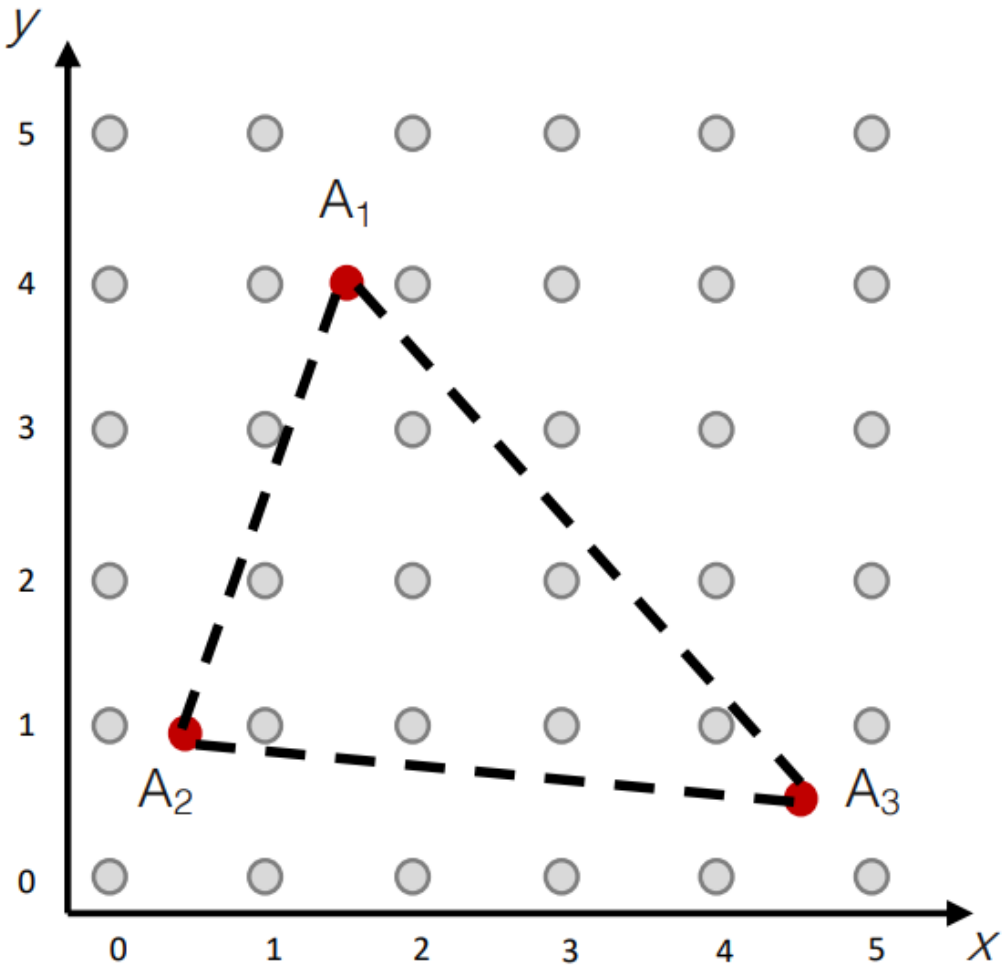
Rasterization



- grid of 6x6 pixels
- 2D vertex positions after transformations

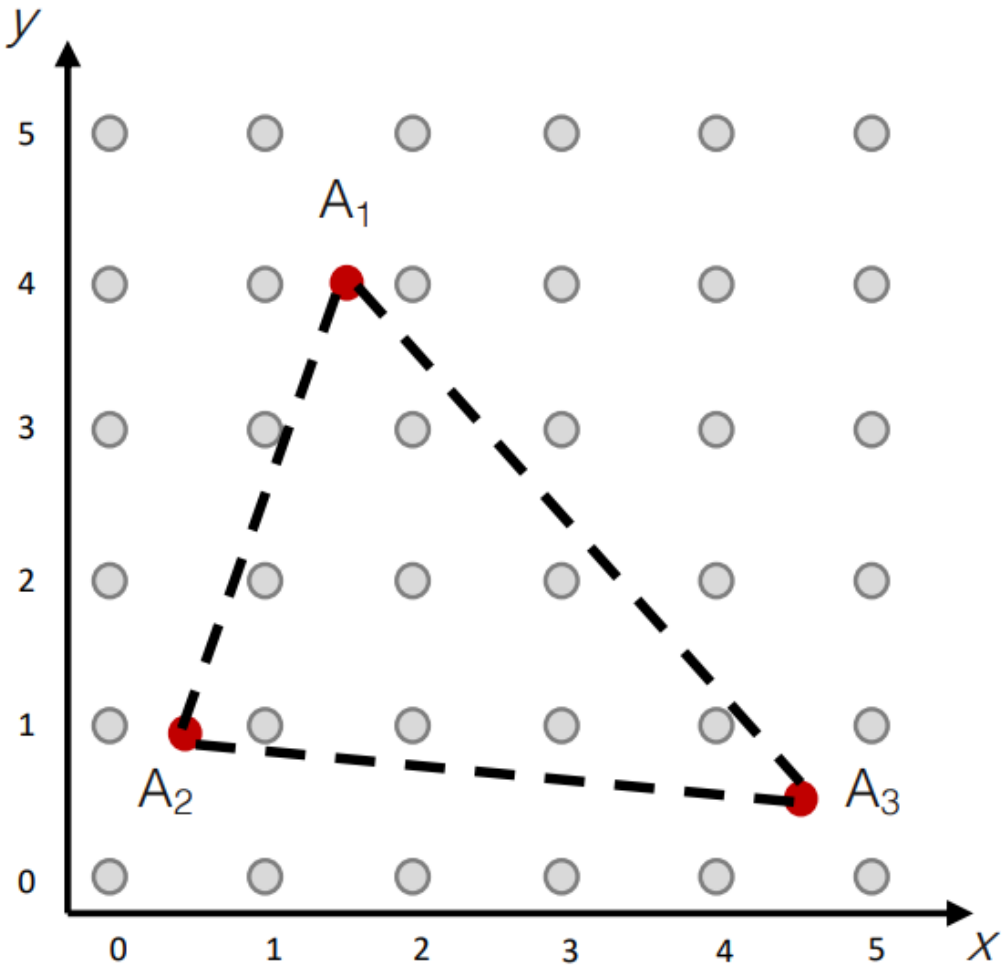
  + edges = triangle

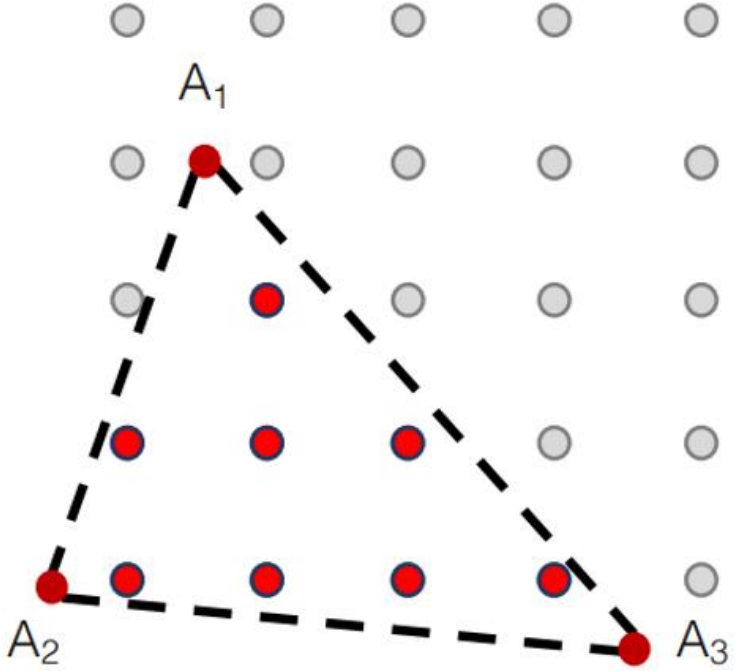
Rasterization



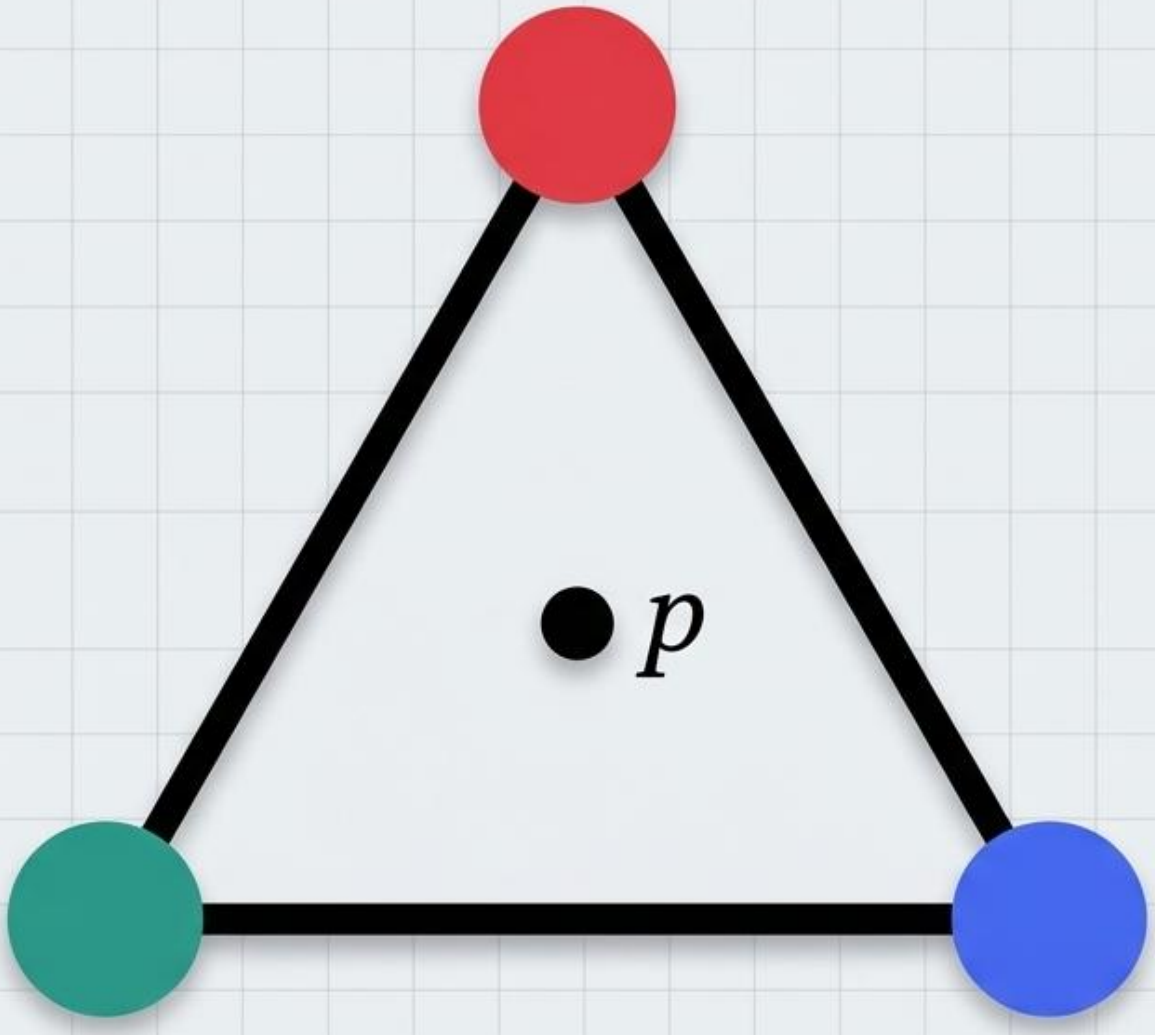
- grid of 6x6 pixels
- 2D vertex positions after transformations + edges = triangle
- each vertex has 1 or more attributes A, such as R/G/B color, depth, …
- user can assign arbitrary attributes, e.g. surface normals

Rasterization





Compute the inside-pixel's attribute

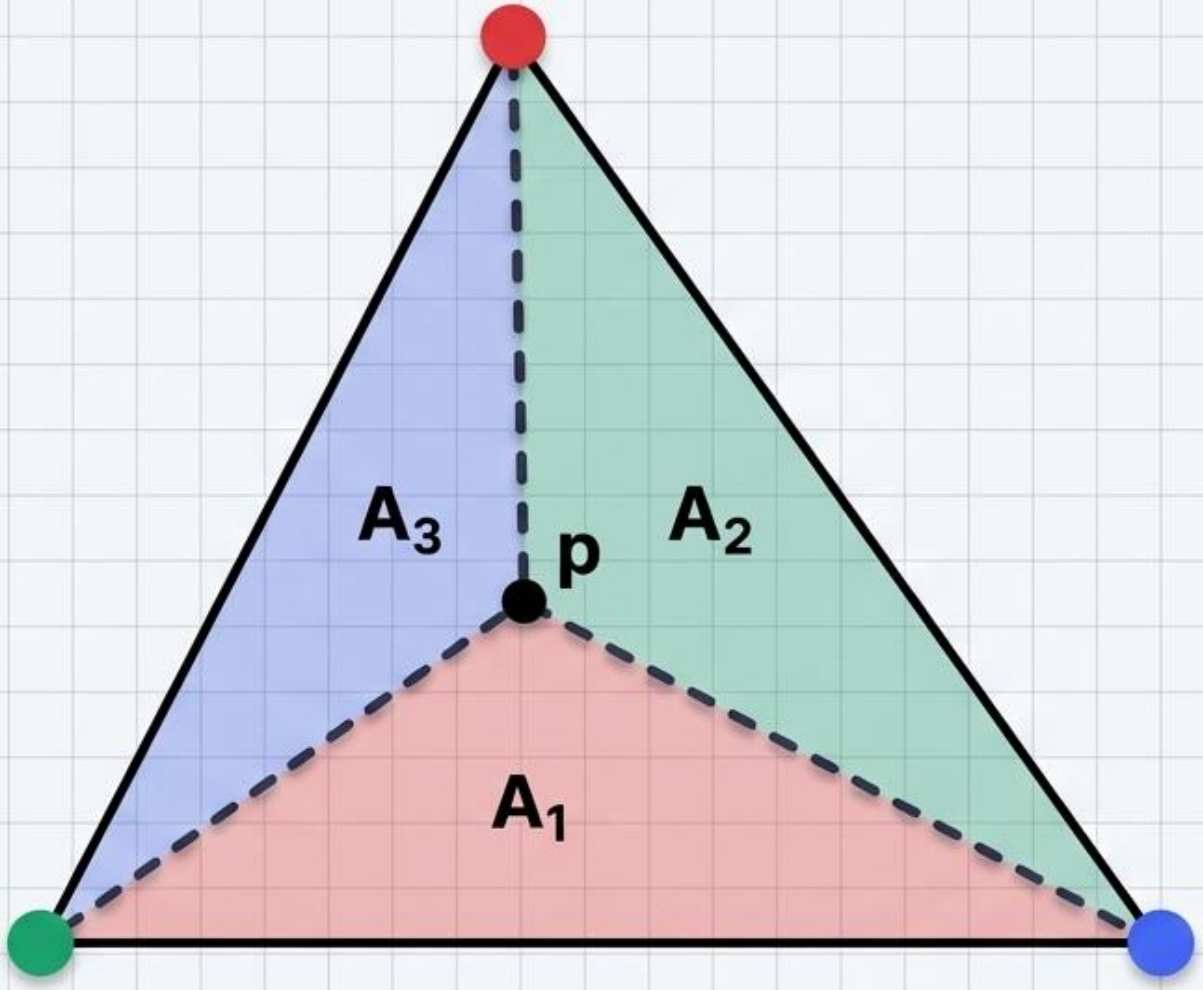


Computing the Inside

내부 픽셀 위치를 찾았다면, 그 픽셀의 색상이나 깊이는 어떻게 정할까요?

정점들이 가진 정보(Attributes)를 거리에 비례하여 섞어줍니다.

→ **Barycentric Interpolation (무게중심 보간)**

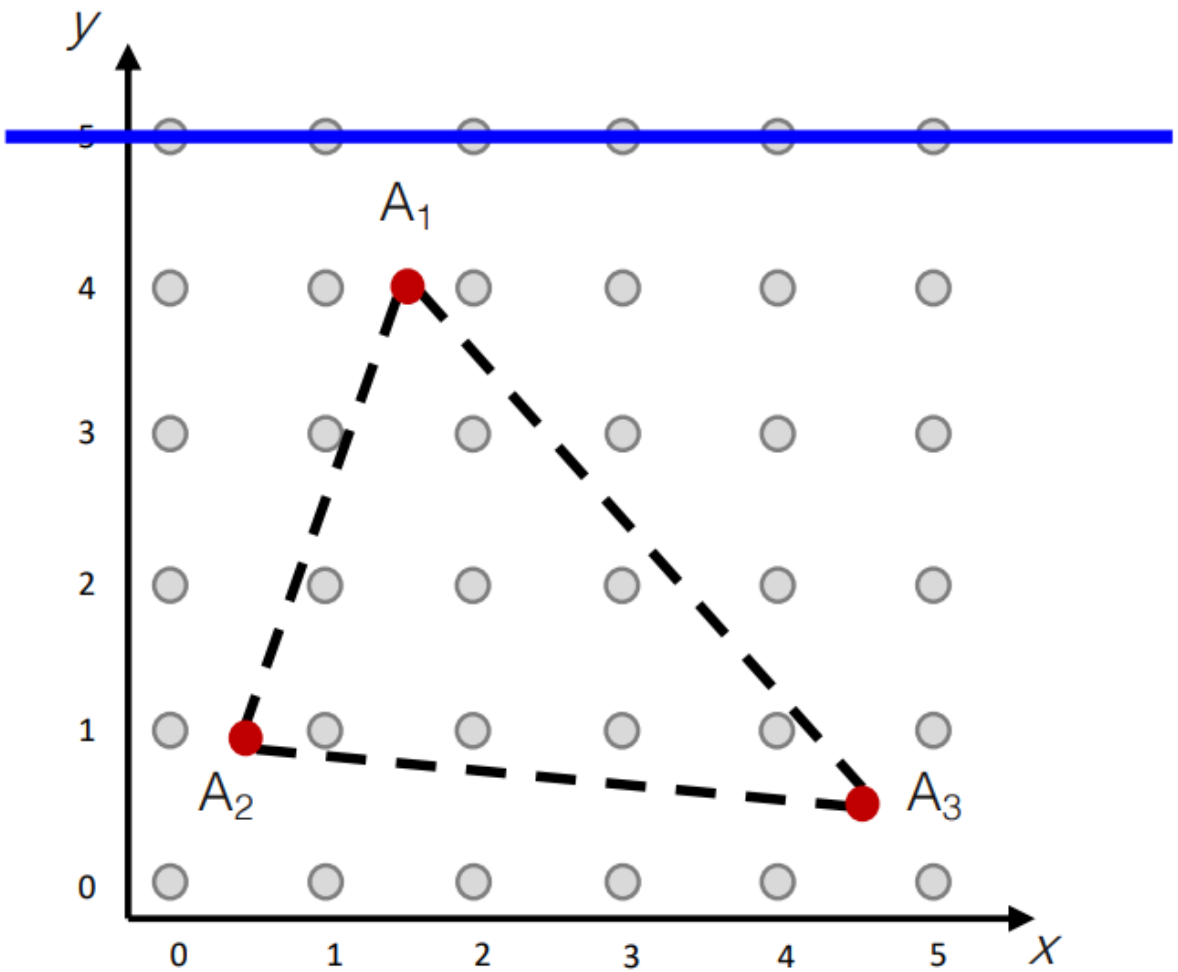


The Mechanics of Barycentric Coordinates

복잡한 수식 대신 '면적의 비율'로 이해해보세요.

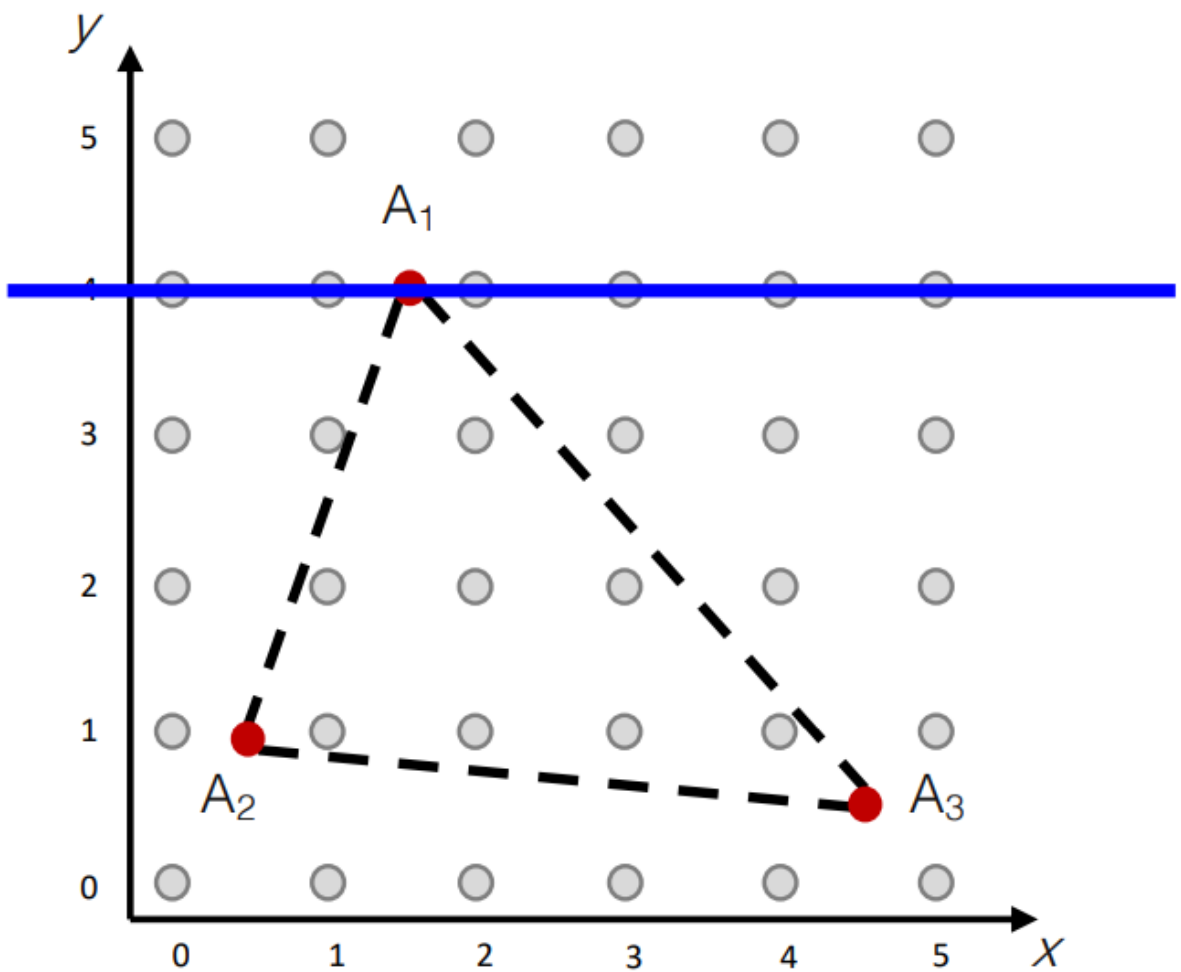
- 점 p가 빨간 정점에 가까울수록, 점 p 반대편에 있는 빨간색 영역(A_1)의 면적이 커집니다.
- 이 면적 비율(λ)이 곧 해당 정점의 속성을 얼마나 가져올지 결정합니다. ($\sum \lambda_i = 1$)

Rasterization / Scanline Interpolation



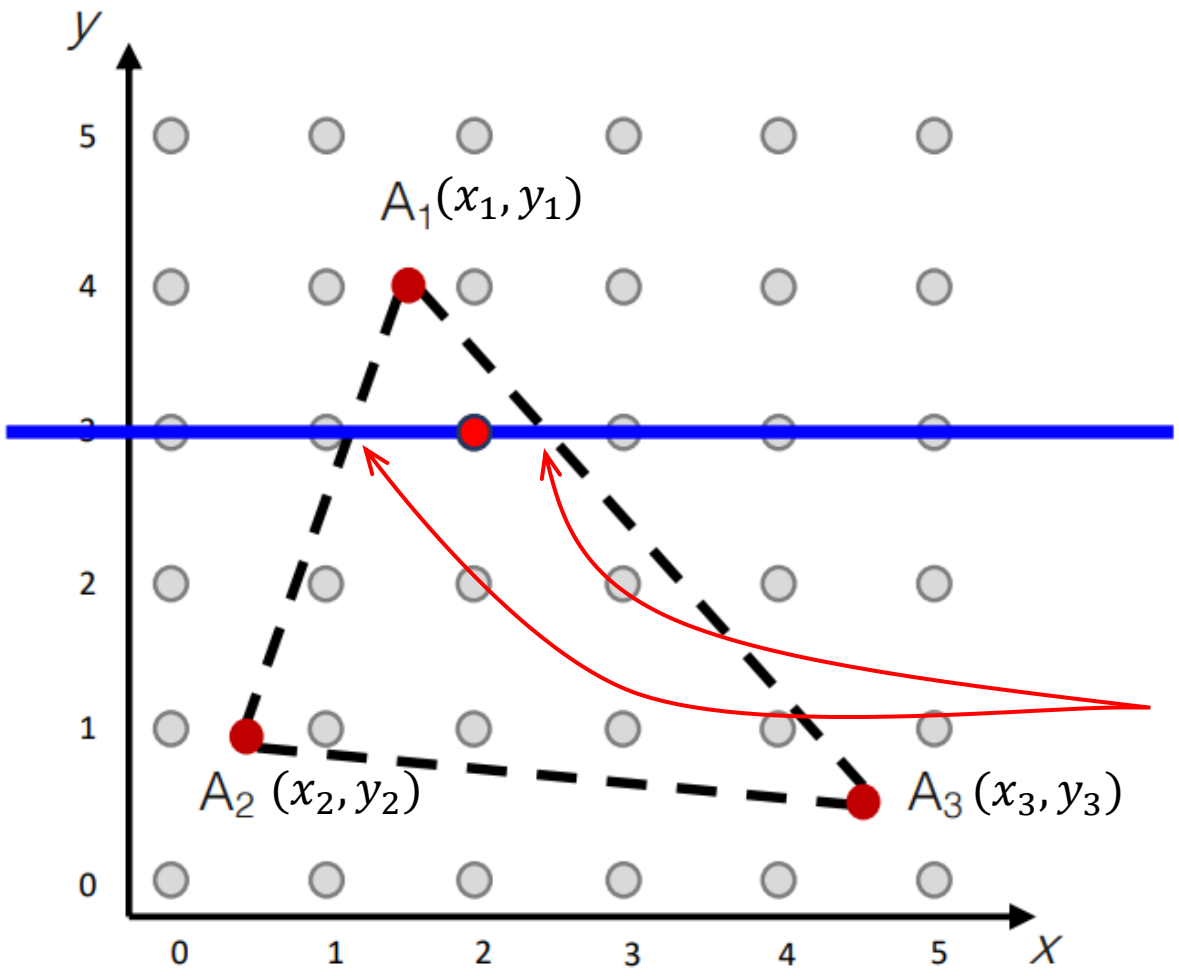
- scanline moving top to bottom

Rasterization / Scanline Interpolation



- scanline moving top to bottom

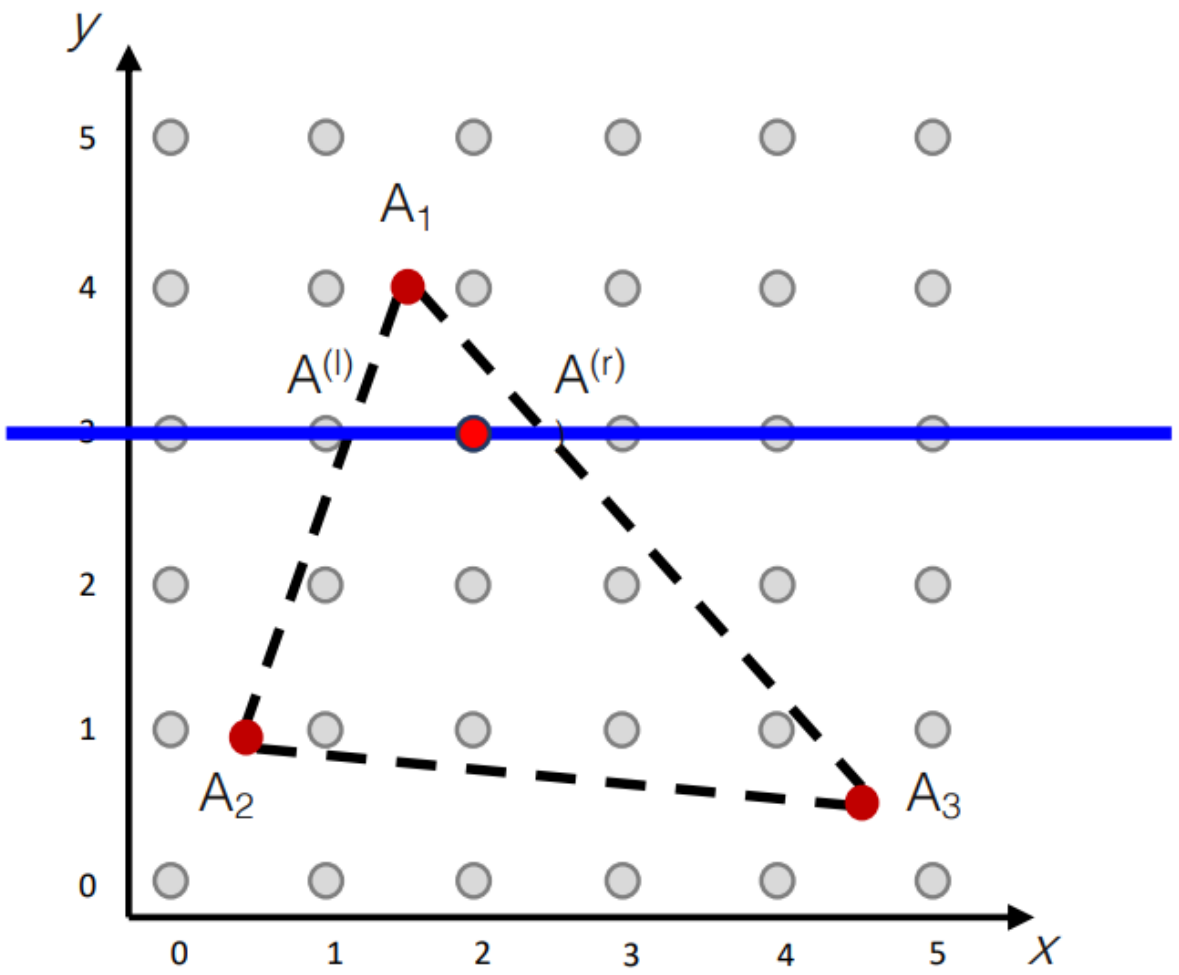
Rasterization / Scanline Interpolation



- scanline moving top to bottom
- determine which pixels are inside the triangle

Q) How can we compute the edge-scanline crossing point?

Rasterization / Scanline Interpolation

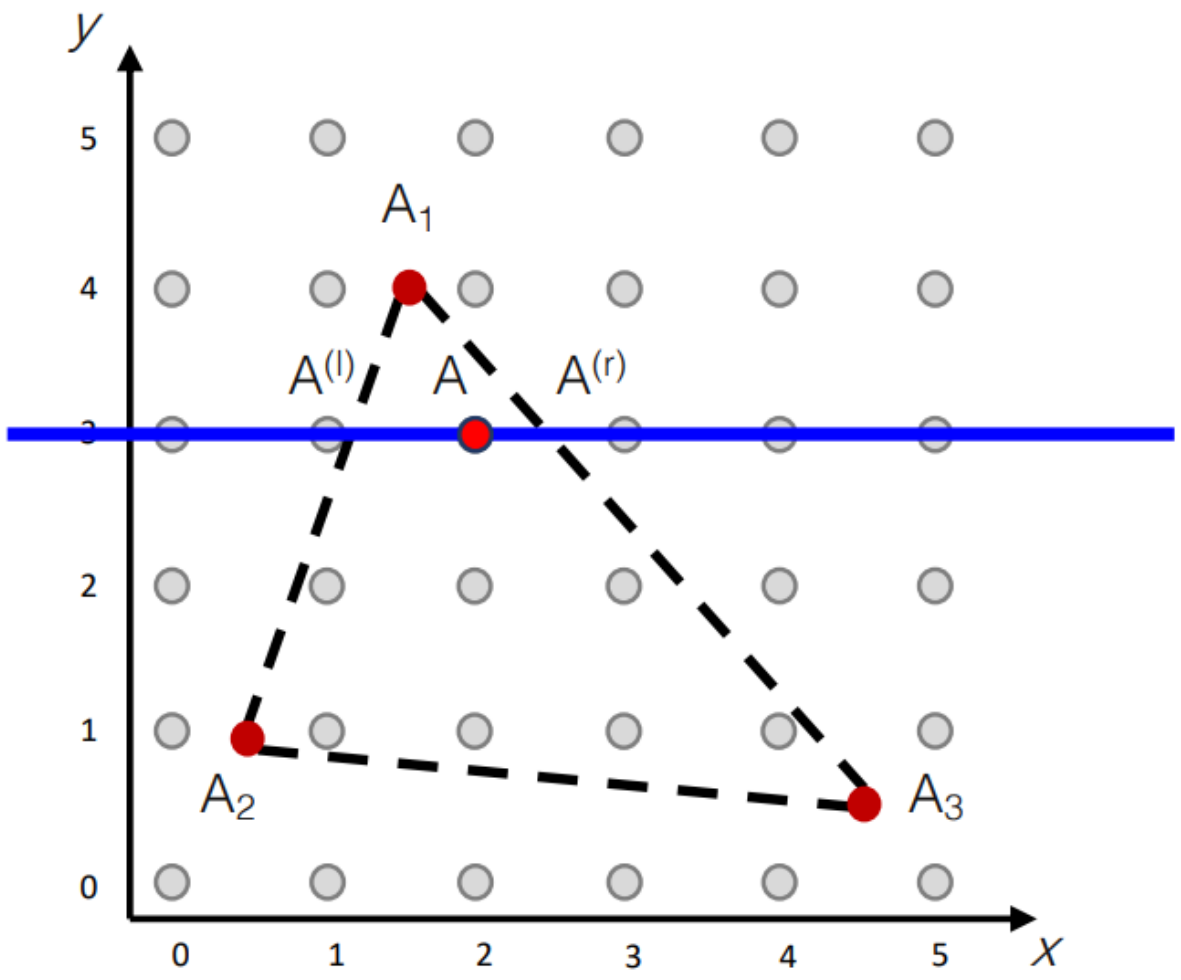


- scanline moving top to bottom
- determine which pixels are inside the triangle
- interpolate attribute along edges in y

$$A^{(l)} = \left(\frac{y^{(l)} - y_2}{y_1 - y_2} \right) A_1 + \left(\frac{y_1 - y^{(l)}}{y_1 - y_2} \right) A_2$$

$$A^{(r)} = \left(\frac{y^{(r)} - y_3}{y_1 - y_3} \right) A_1 + \left(\frac{y_1 - y^{(r)}}{y_1 - y_3} \right) A_3$$

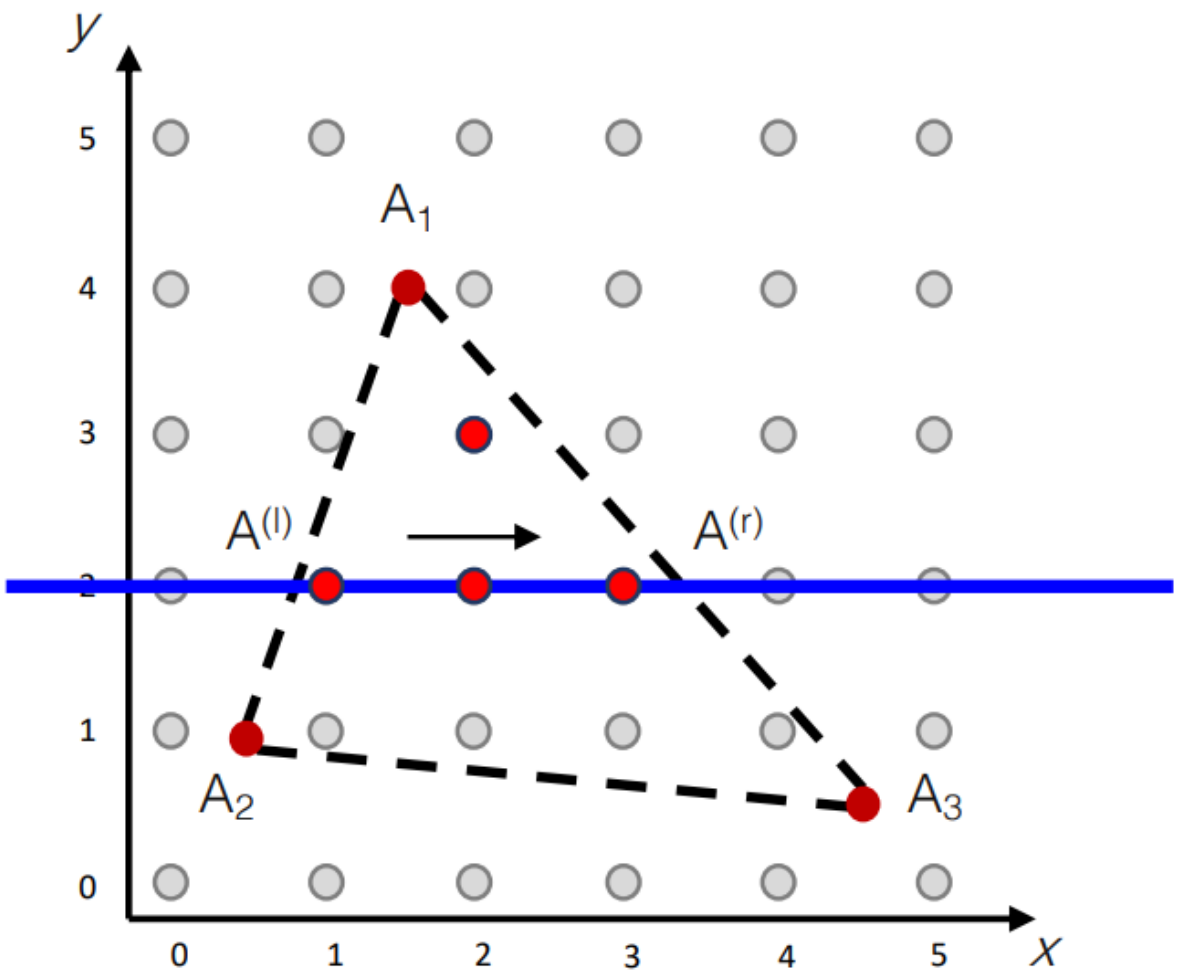
Rasterization / Scanline Interpolation



- scanline moving top to bottom
- determine which pixels are inside the triangle
- interpolate attribute along edges in y
- then interpolate along x

$$A = \left(\frac{x - x^{(l)}}{x^{(r)} - x^{(l)}} \right) A^{(r)} + \left(\frac{x^{(r)} - x}{x^{(r)} - x^{(l)}} \right) A^{(l)}$$

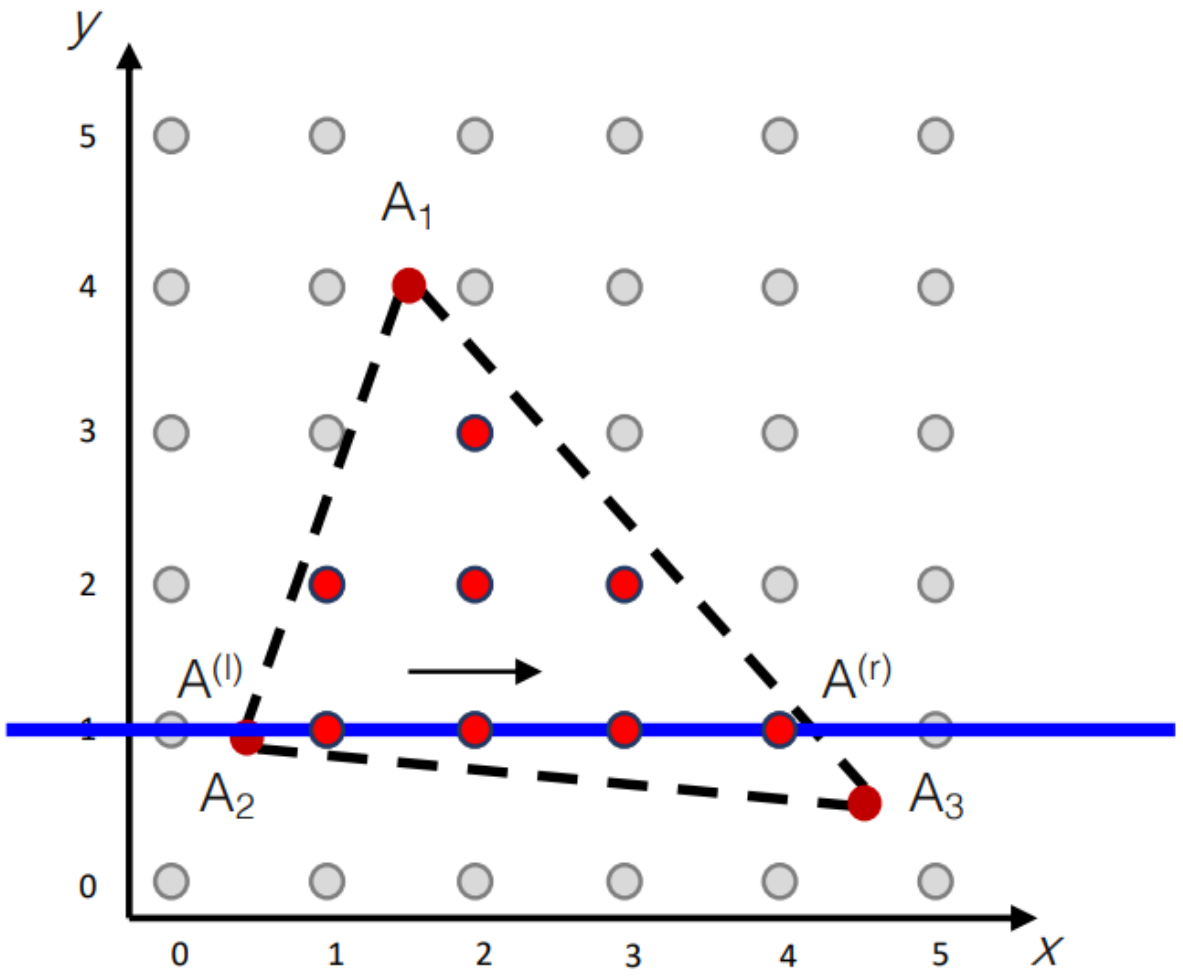
Rasterization / Scanline Interpolation



repeat:

- interpolate attribute along edges in y
- then interpolate along x

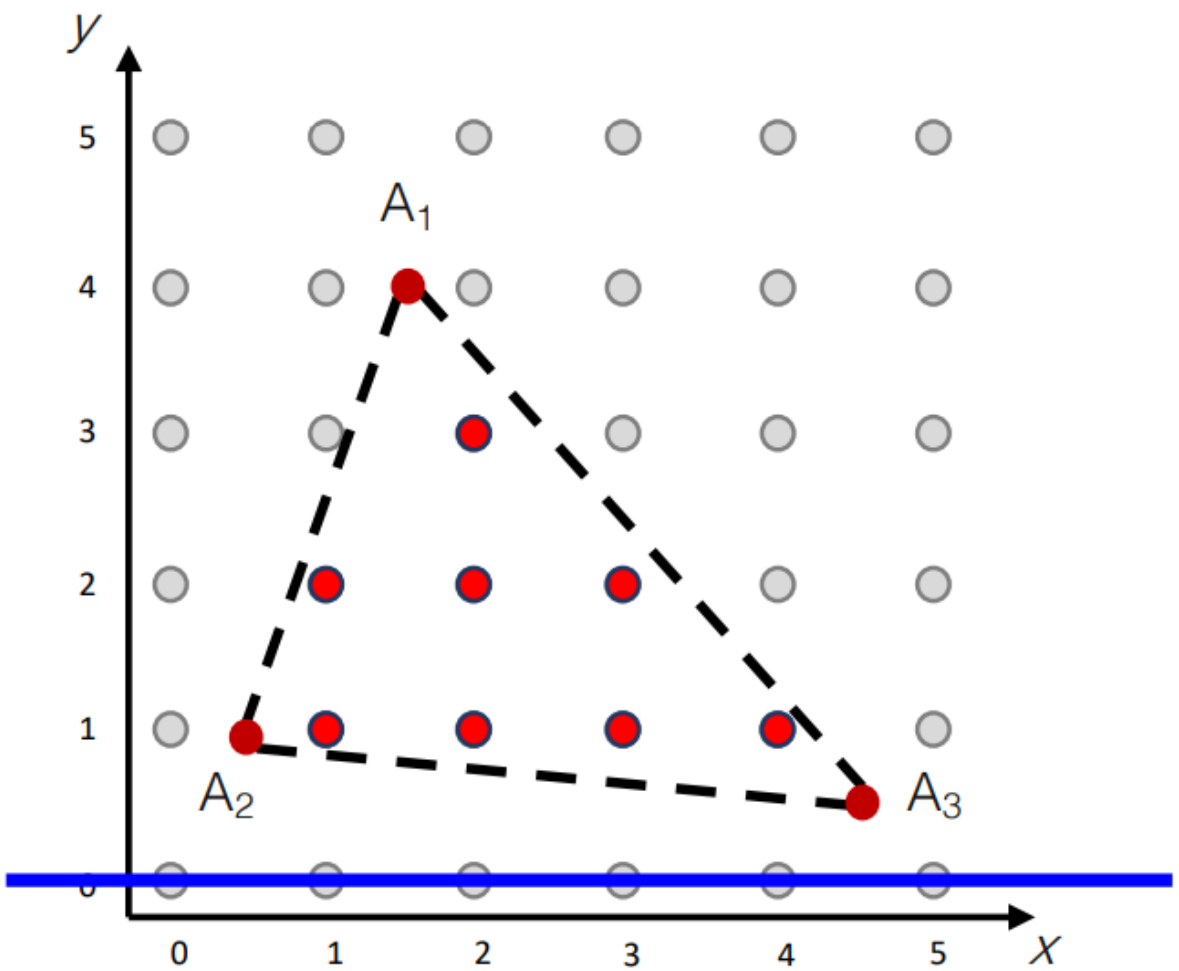
Rasterization / Scanline Interpolation

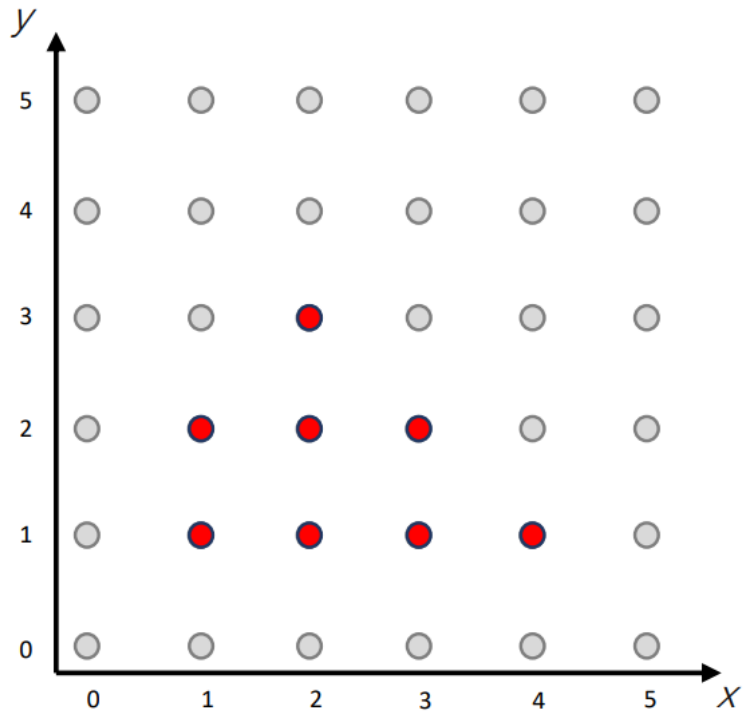


repeat:

- interpolate attribute along edges in y
- then interpolate along x

Rasterization / Scanline Interpolation



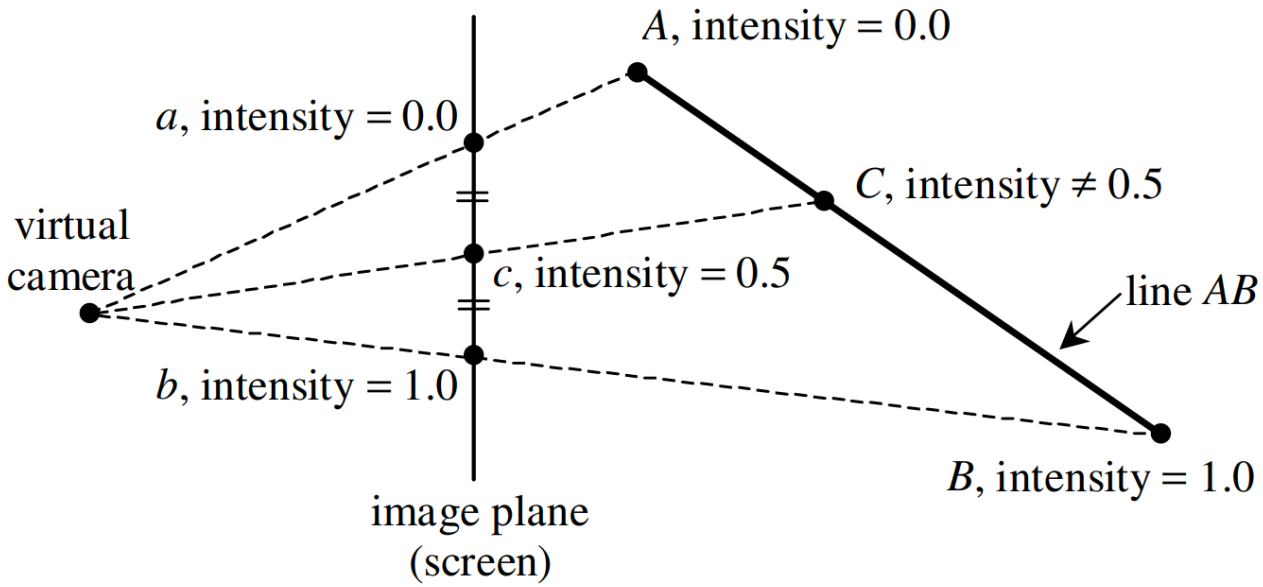


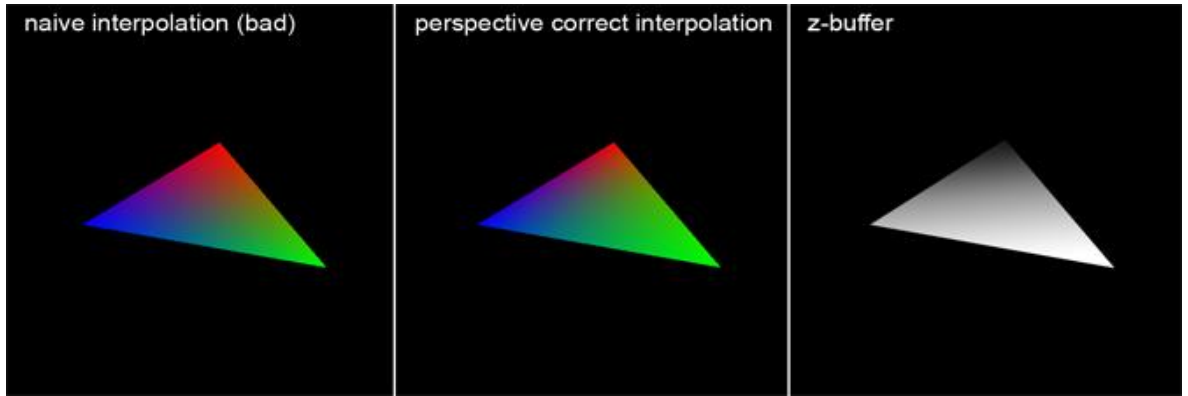
output: set of pixels inside triangle(s) with interpolated attributes for each of these fragments

Note: the inside-pixels can be computed in parallel!

Perspective-Correct Interpolation

- Note the linear interpolation of attribute values in the screen space does not guarantee depth-corrected linear interpolation results





Perspective-Correct Interpolation

$$s : 1 - s \neq t : 1 - t$$

$$\frac{X_1}{Z_1} = \frac{u_1}{d} \Rightarrow X_1 = \frac{u_1 Z_1}{d},$$

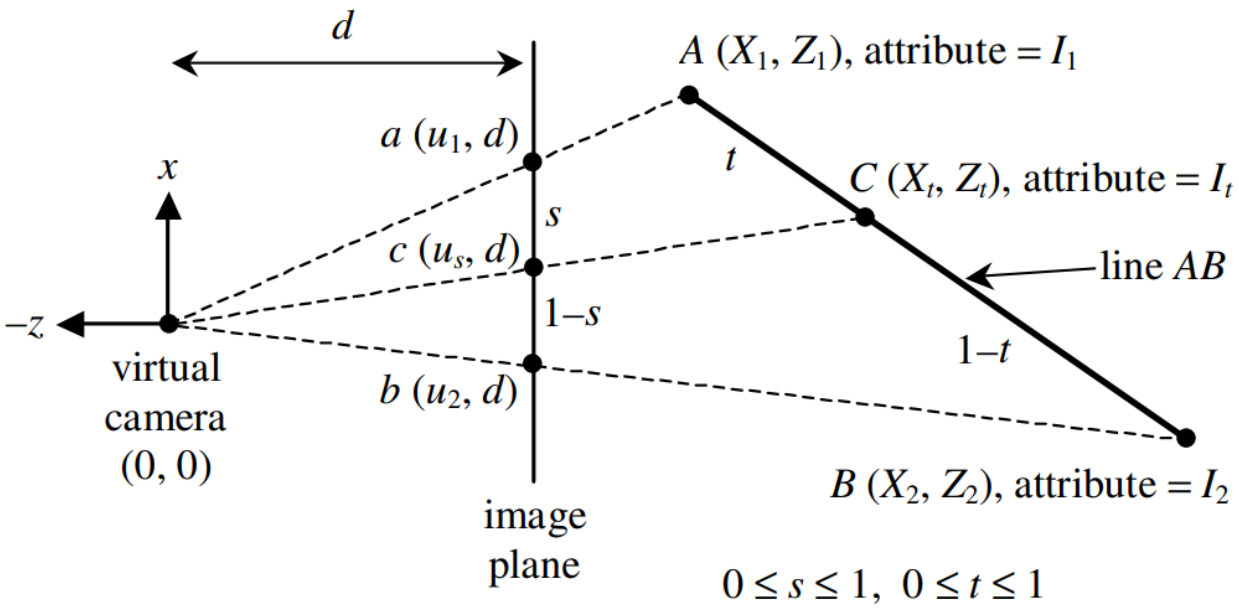
$$\frac{X_2}{Z_2} = \frac{u_2}{d} \Rightarrow X_2 = \frac{u_2 Z_2}{d},$$

$$\frac{X_t}{Z_t} = \frac{u_s}{d} \Rightarrow Z_t = \frac{dX_t}{u_s}.$$

$$t = \frac{sZ_1}{sZ_1 + (1-s)Z_2}$$

$$I_t = (1-t)I_1 + tI_2$$

$$I_t = \frac{(1-s)\frac{I_1}{Z_1} + s\frac{I_2}{Z_2}}{(1-s)\frac{1}{Z_1} + s\frac{1}{Z_2}} \Rightarrow \frac{1}{Z_t}$$

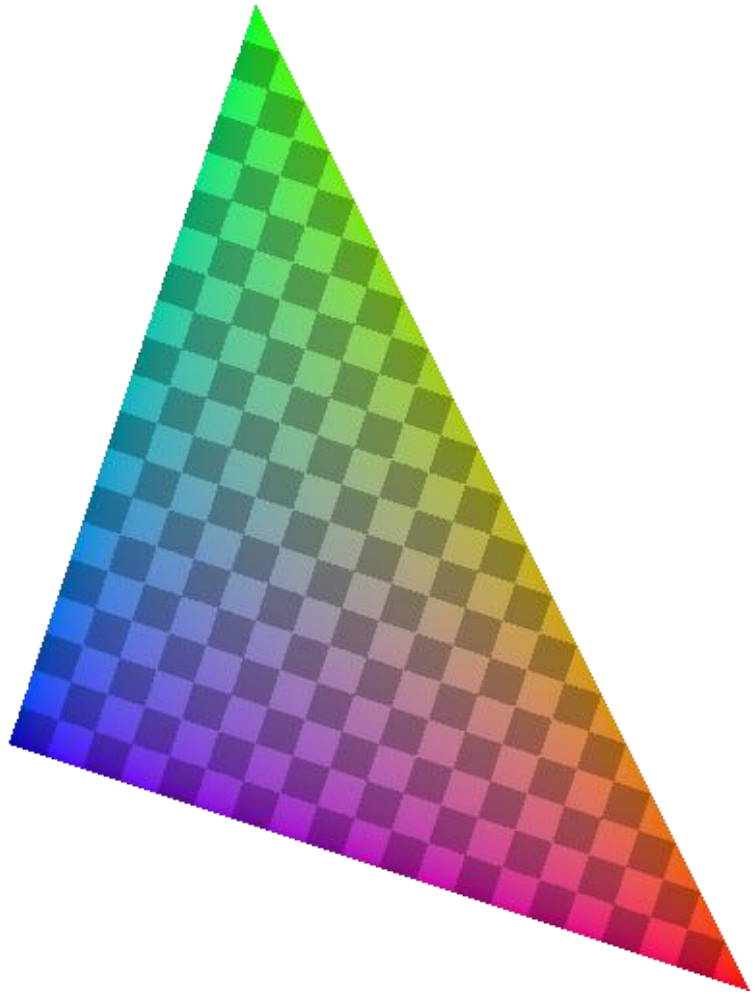


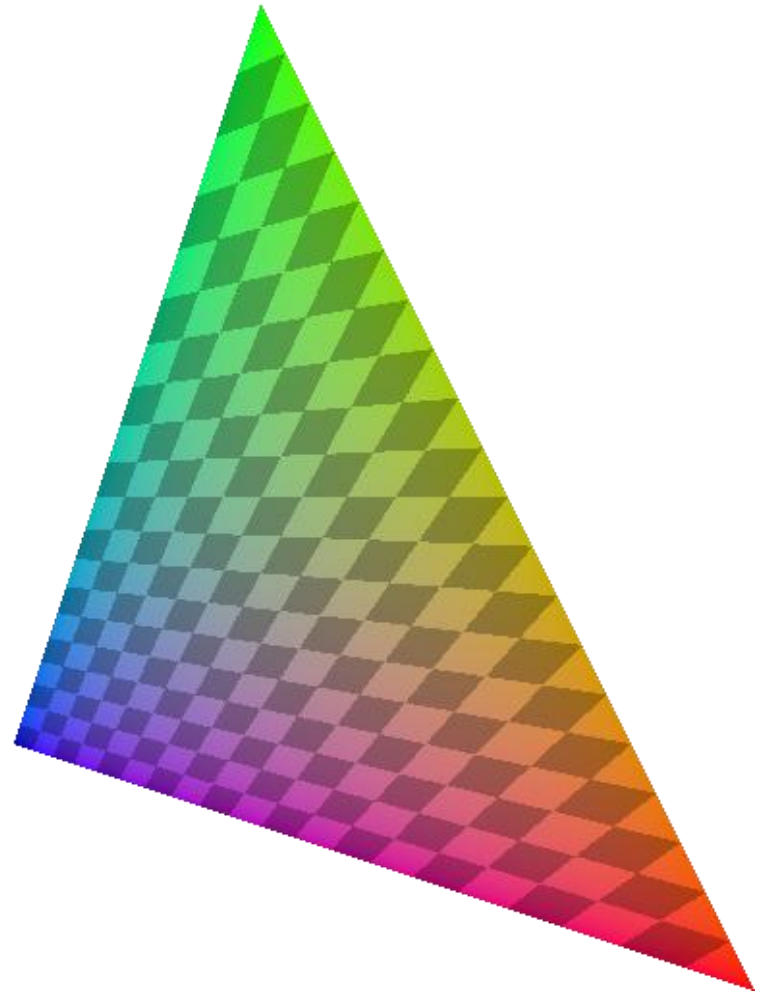
This implies that the attribute value at point c in the image plane (screen space) can be corrected by just linearly interpolating between $\frac{I_1}{Z_1}$ and $\frac{I_2}{Z_2}$ and then divide the interpolated result of $\frac{1}{Z_t}$

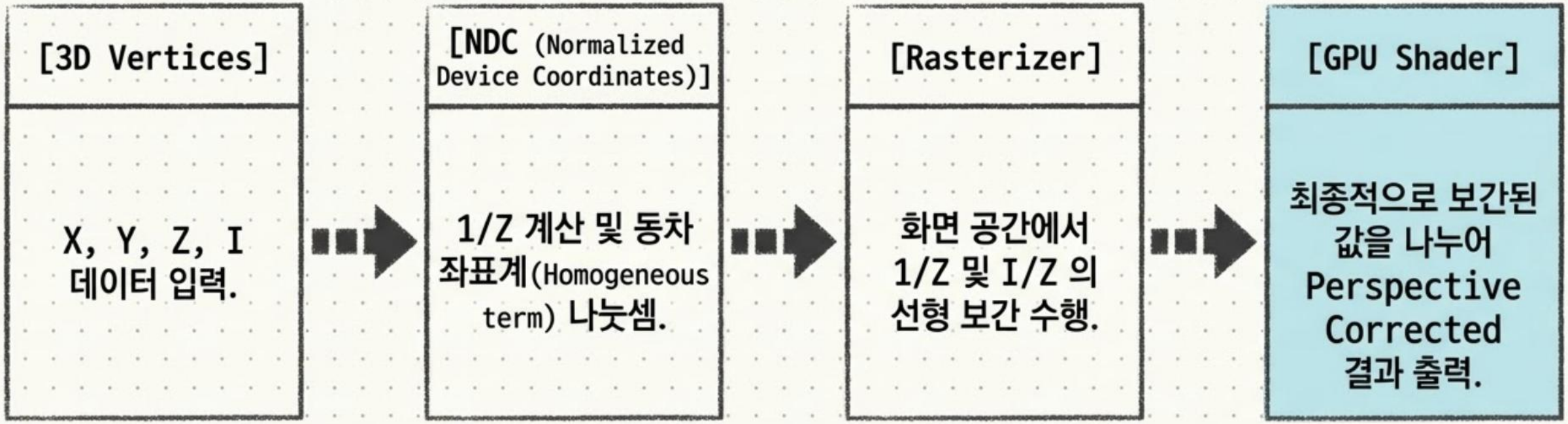
Simply implemented in the graphics pipeline

- Dividing the attribute value by the homogeneous term in the projection space (or NDC)
- GPU Shader provides the perspective corrected interpolation results


3D
Rasterizer
3D





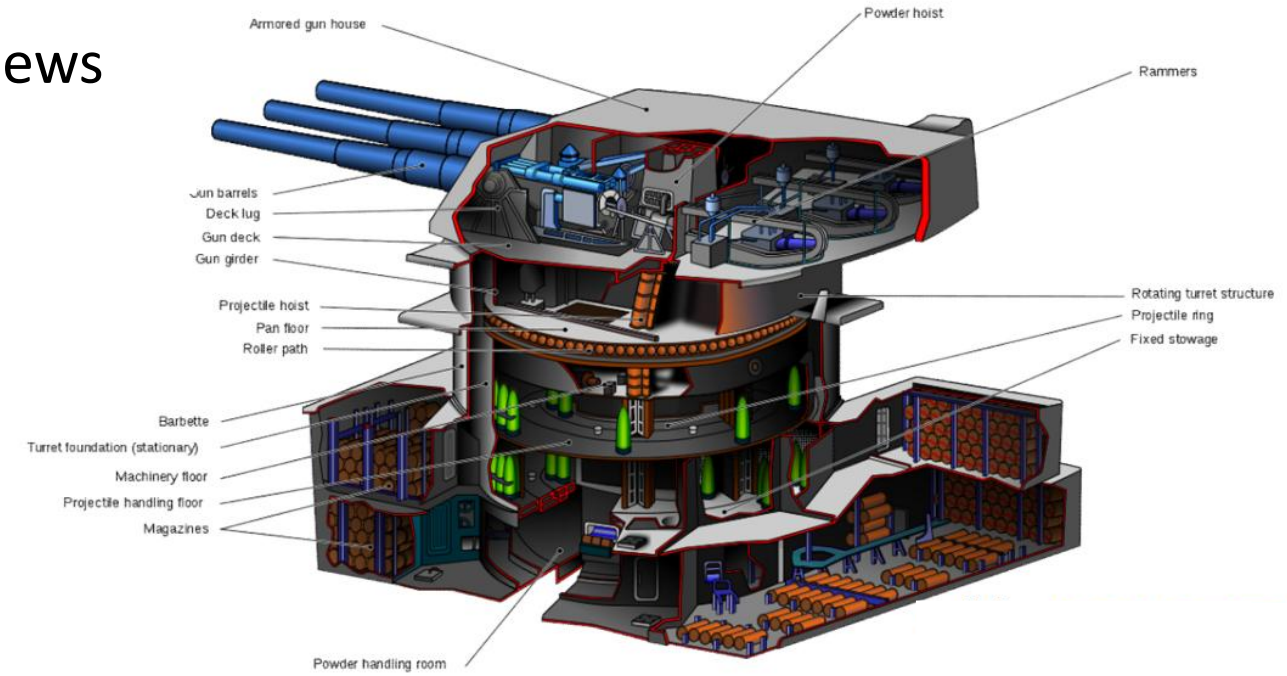


최신 그래픽스 파이프라인에서 이 수학적 보정 과정은 하드웨어 수준에서 매우 빠르고 단순하게(`Simply implemented`) 처리되며, `GPU Shader` 를 통해 최종적으로 왜곡 없는 완벽한 렌더링 결과를 제공합니다.

Culling and Clipping

Culling and Clipping

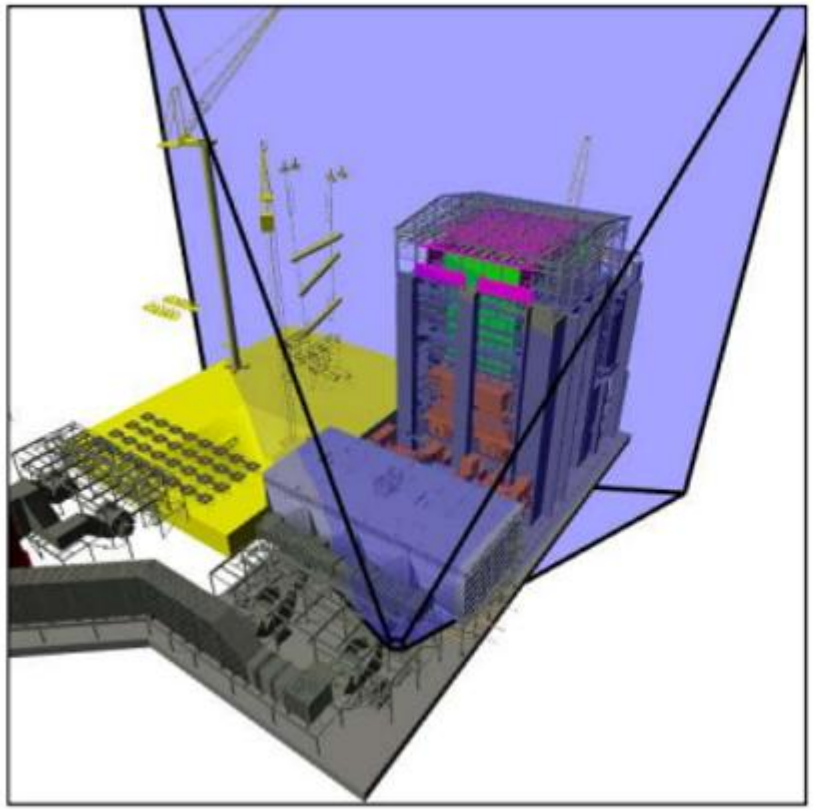
- Culling
  - Throws away entire objects and primitives that cannot possibly be visible
  - An important rendering optimization (e.g., open world)
- Clipping
  - “Clips off” the visible portion of a primitive
  - Simplifies rasterization
  - Also, used to create “cut-away” views



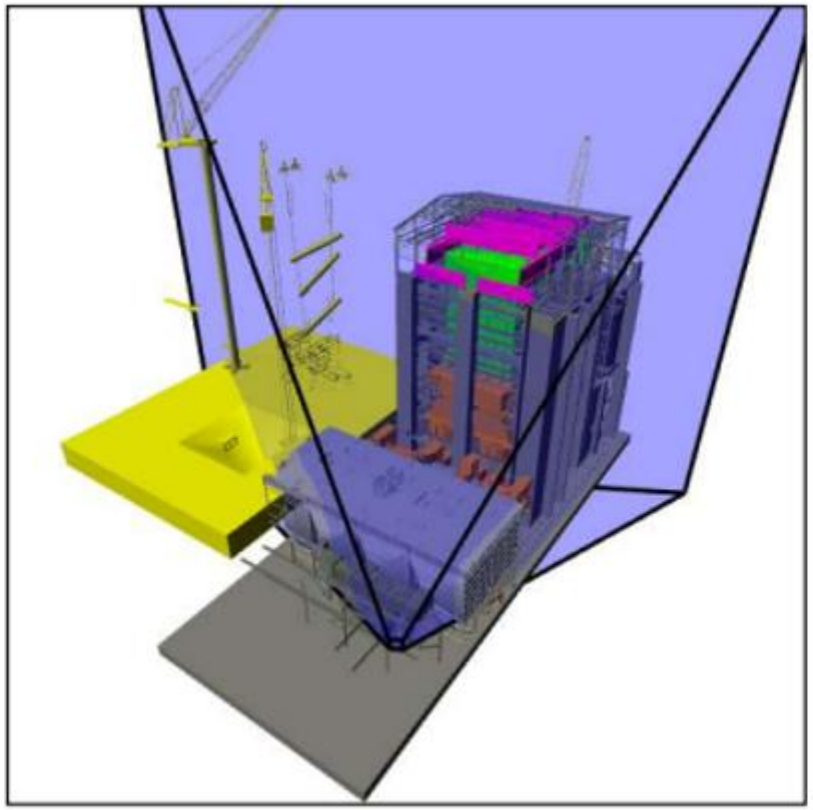
Culling Example

Power plant model
(12 million triangles)

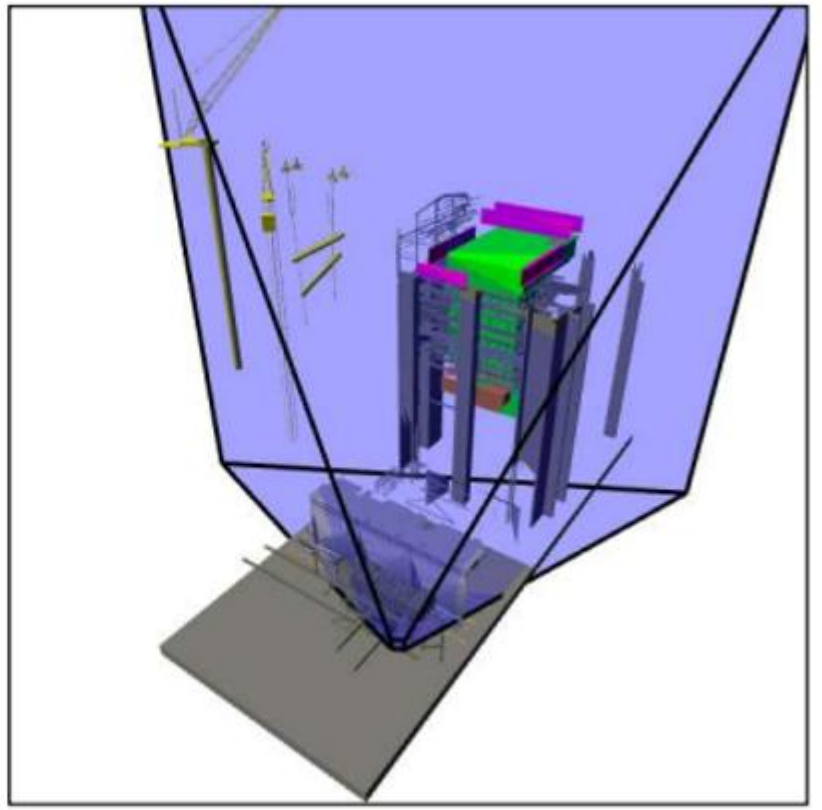
Culling Example



Full model
12 Mtris



View frustum culling
9 Mtris

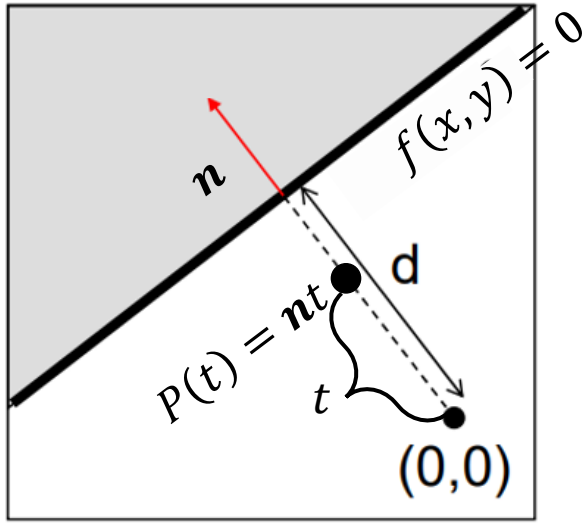


Occlusion culling
1 Mtris

1st Question. How do we exclude the geometries outside the view frustum?

Algebra: Lines and Planes

- When $\boldsymbol{n} = (n_x, n_y, n_z)$ is a normal vector
- Implicit function for line (e.g., a line on XY plane)
  - $f(x, y) = n_x x + n_y y - d = 0$
  - $\boldsymbol{n} = (n_x, n_y)$ is a normal vector of a 2D line
- Implicit function for plane
  - $f(x, y, z) = n_x x + n_y y + n_z z - d = 0$
  - $\boldsymbol{n} = (n_x, n_y, n_z)$ is a normal vector of a 3D plane
- If $\boldsymbol{n}$ is a unit vector, then d gives the distance of the line (plane) from the origin point along the normal vector $\boldsymbol{n}$
  - Can you prove this? ☺



Algebra: Implicit vs Explicit function

- **Explicit function**
  - **One variable is clearly expressed as a function of other variables**
  - e.g., $y = f(x) = 3x^2 - 5$
  - **Food for thoughts: Variable vs. Parameter**
- **Implicit function**
  - All variables are '***implied***' by a function
  - e.g., $f(x, y) = (x - 3)^2 + (y - 1)^2 = 9$ (we express this by two explicit functions)

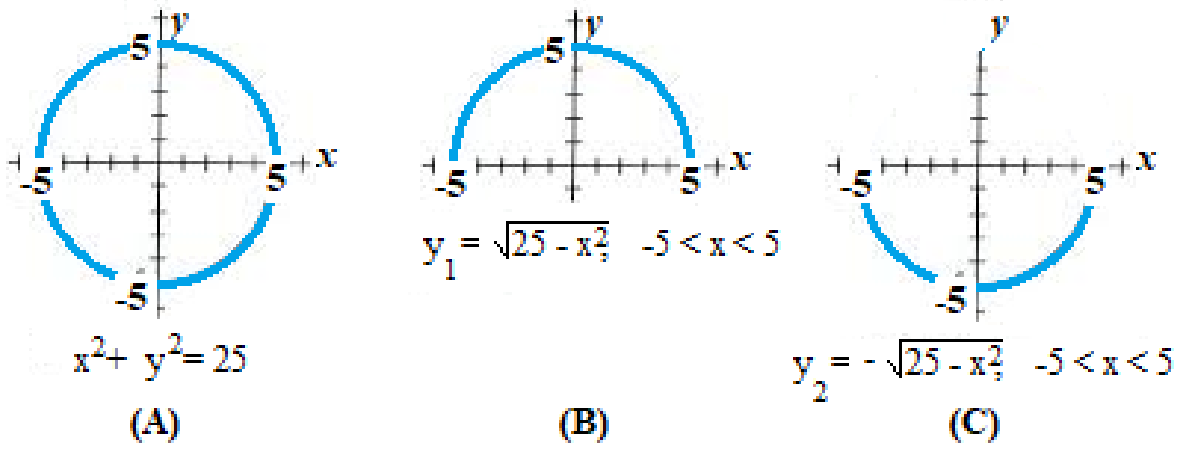
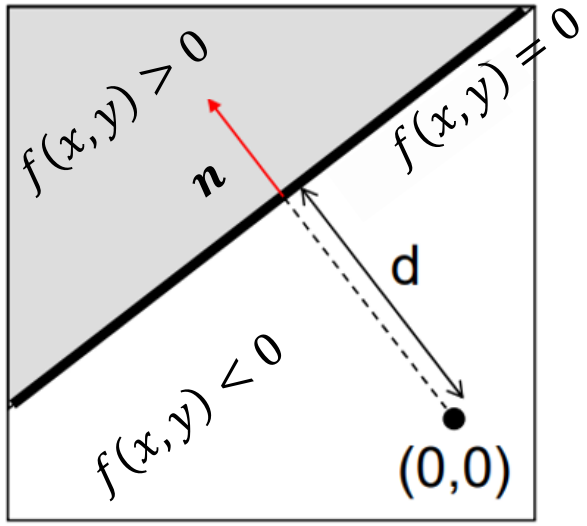
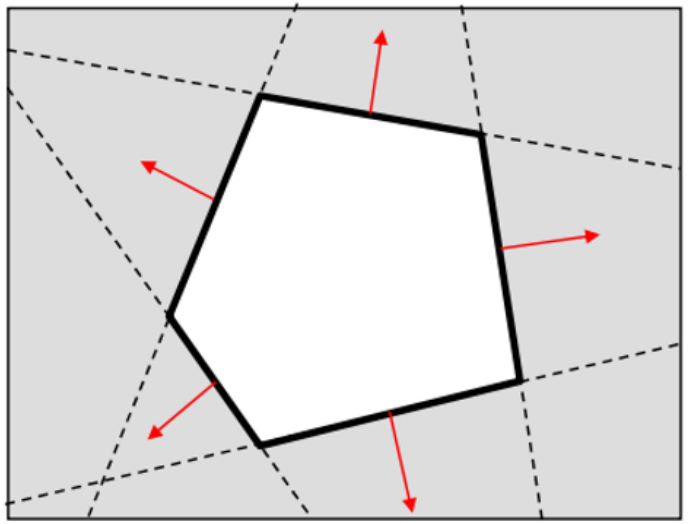


Image: https://www.calculushowto.com/explicit-function-implicit-solution/

Algebra: Lines and Planes

- Lines (planes) partition 2D (3D) space
  - Positive and negative half-spaces
- The intersection of negative half-spaces defines a convex region



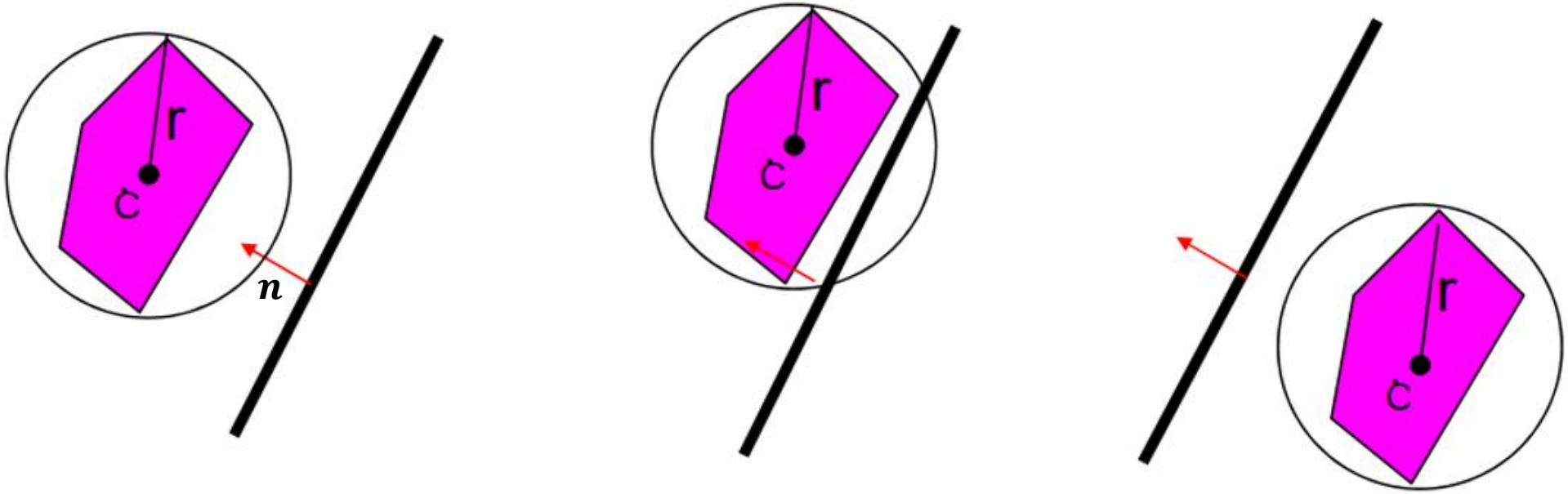


Conservative Testing

- Use cheap, conservative bounds for trivial cases
- We can use more accurate, more expensive tests for ambiguous cases if needed

$f(x, y) = n_x x + n_y y - d = 0$

$\boldsymbol{n} = (n_x, n_y)$ is a normal vector of a 2D line



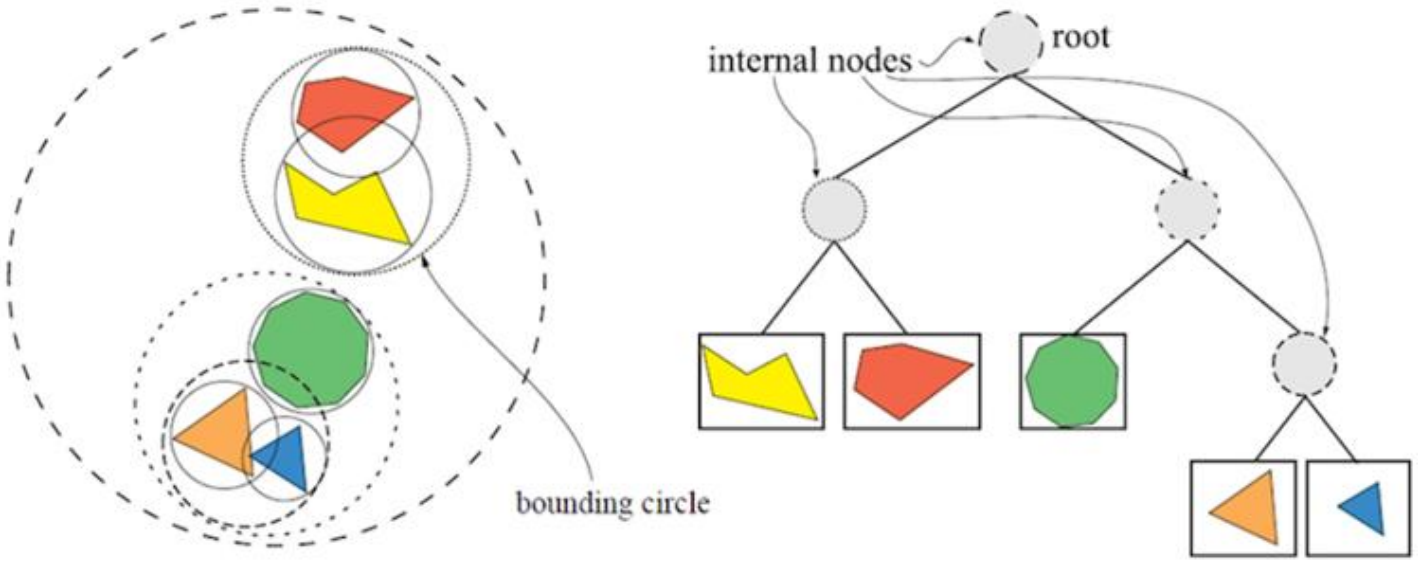
Outside
$f(x, y) > r$

Indeterminate
$-r < f(x, y) < r$

Inside
$-r < f(x, y)$

Hierarchical Culling (structure)

- Bounding volume hierarchies (BVHs)
  - Accelerate culling by rejecting / accepting entire subtrees at a time
  - Uses very simple bounding primitives to determine the in/outside
    - e.g., axis-aligned bounding boxes (simplest), spheres, …
  - Also known as object partitioning hierarchies

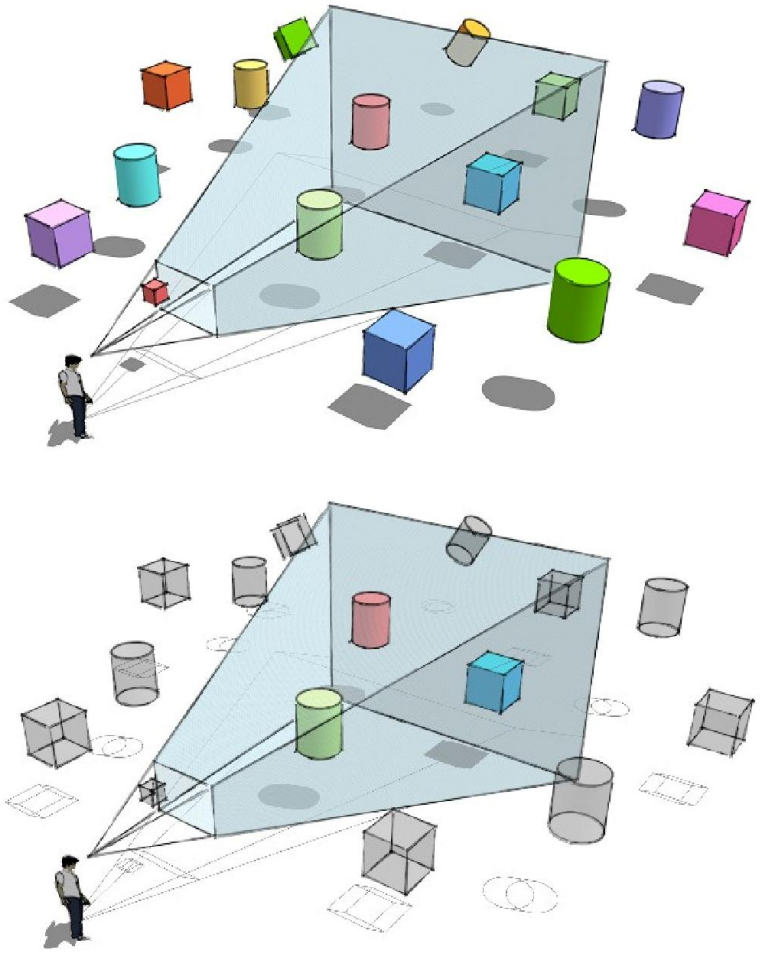


Traversing a BVH (i.e., tracing a ray) is typically **O(log N)**.

Image courtesy of
Real-Time Rendering

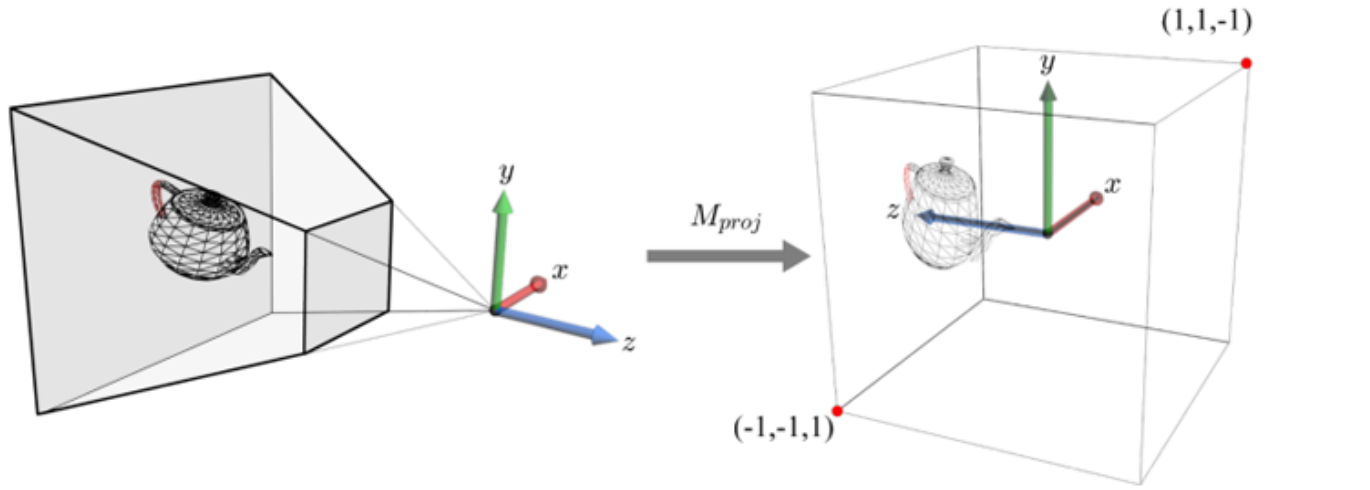
View Frustum Culling (target)

- In rendering, the objects inside the view frustum are visible
  - The outside objects must be rejected (i.e., culled) for fast rendering



How do we compute them?

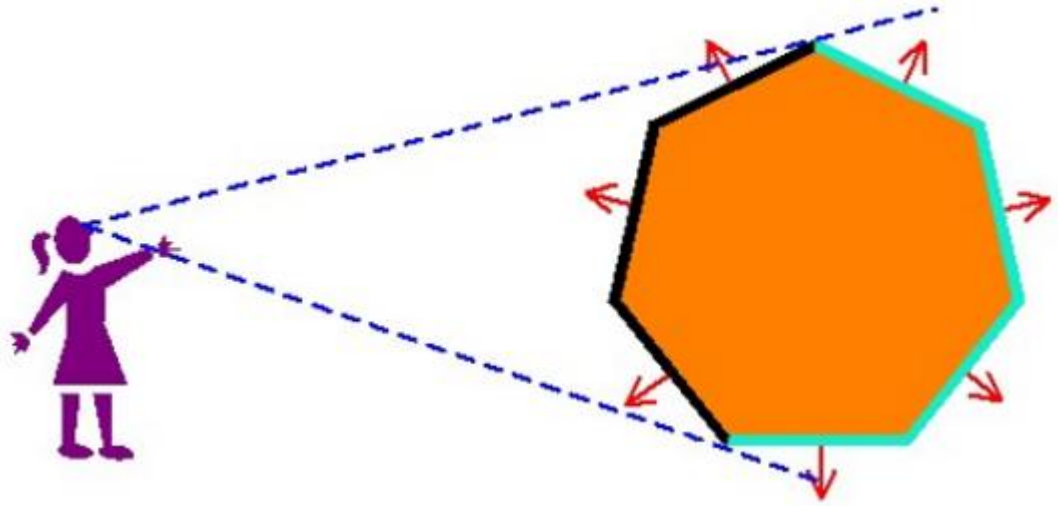
Direct3D: (1, 1, 0)
OpenGL: (1, 1, -1)

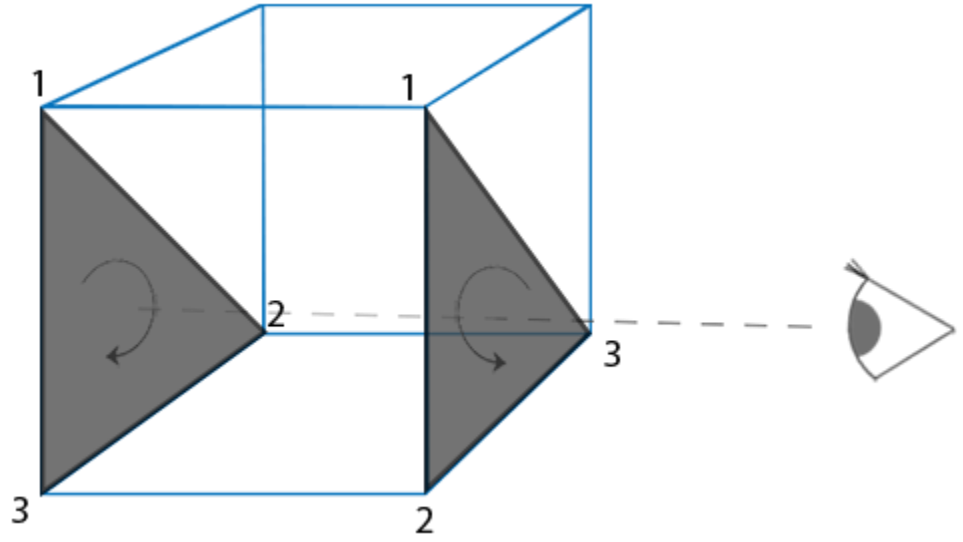


Transform them into the '***easy***' space

Back-face Culling (target)

- The backside (back-face) of the object is not visible
  - Opaque surface case (not transparency rendering)
- How do we determine if the primitive (triangle) is the back-face?
  - Use ‘normal’ vector of the triangle
  - Use the direction of the triangle (CCW or CW)





Threejs Setting for Culling

THREE.Object3D

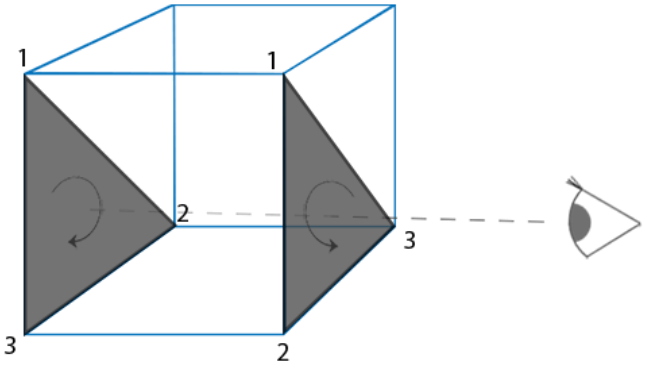
.frustumCulled : Boolean

When this is set, it checks every frame if the object is in the frustum of the camera before rendering the object. If set to `false` the object gets rendered every frame even if it is not in the frustum of the camera. Default is `true`.

THREE.Material

.side : Integer

Defines which side of faces will be rendered - front, back or both. Default is THREE.FrontSide. Other options are THREE.BackSide and THREE.DoubleSide.



How do we cull out the back-side triangle?

Culling Summary

- Goal is to reject the invisible objects in the rendering pipeline
- First culling: cull out the invisible objects before processing the GPU pipeline (object level)
  - View frustum and occlusion culling
- Second culling: cull out the invisible faces (GPU-level)
- Third culling: cull out the occluded fragments (GPU-level)

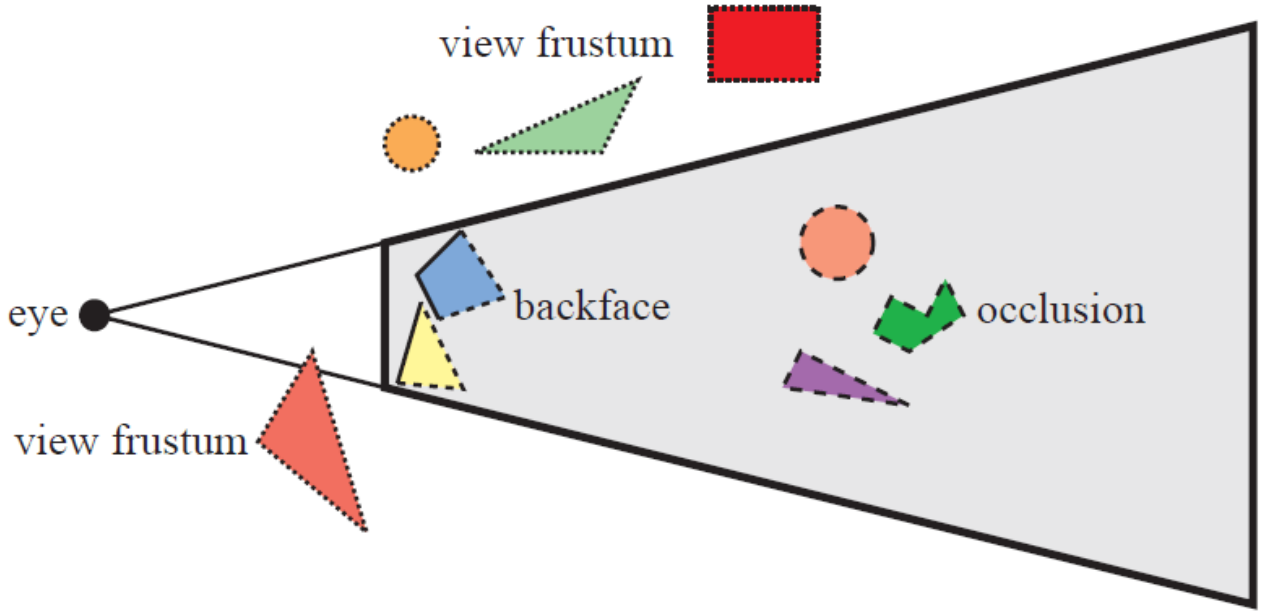
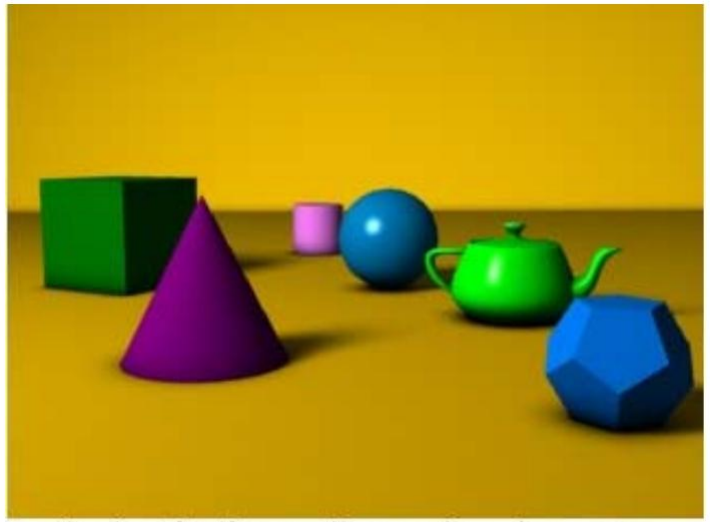


Image: https://bruop.github.io/frustum_culling/

Z-Buffering

- Occlusion culling
  - Primitives occluded by opaque geometry must not be visible
  - Fragments closest to eye along the z-direction must be visible
- Z-buffer (i.e., depth buffer) is used to resolve the visibility test
  - Stores the (smallest) depth values at each pixel
  - Test: the incoming fragment's depth is less than the stored depth at the same pixel
- Z-value range is determined by the View Port (normally set to [-1, 1])
- Z-value is treated as a vertex attribute during the rasterization (gl_position)



A simple three dimensional scene

Z-buffer representation

Z-buffer (Depth buffer) 시각화

A simple three dimensional scene
(최종 컬러 렌더링)

Z-buffer representation
(메모리에 저장된 깊이 맵)

- Z-value Range: View Port에 의해 결정되며, 보통 [-1, 1] 또는 [0, 1] 범위를 가집니다.
-
-

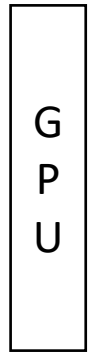
Culling and Clipping in Graphics Pipeline

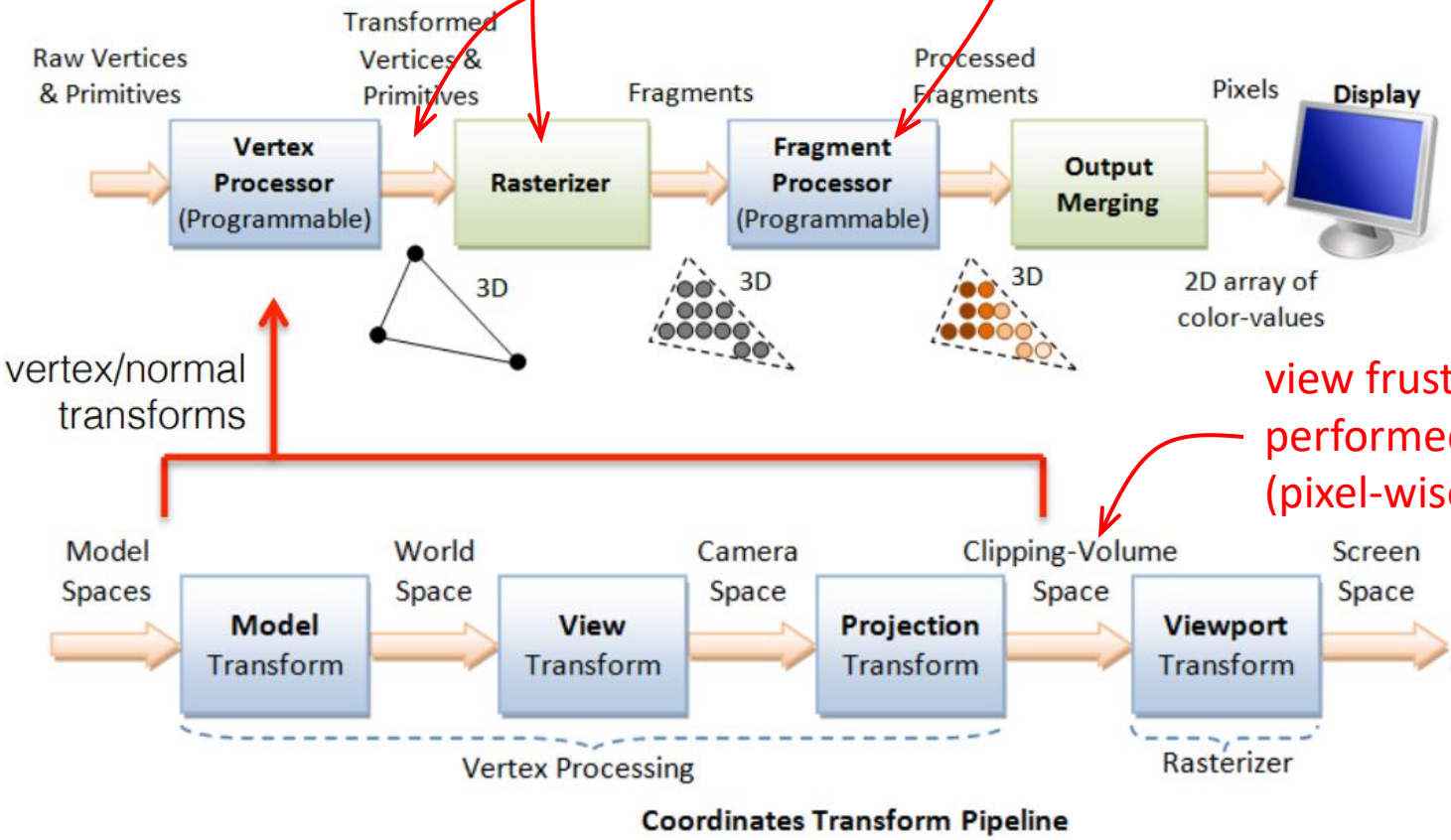
view frustum culling
and occlusion culling
performed in the application level
(object-wise)

occlusion culling by z-test

back-face culling

Scene models





https://www.ntu.edu.sg/home/ehchua/programming/opengl/CG_BasicsTheory.html

Questions? ☺

See You Next Class!!